

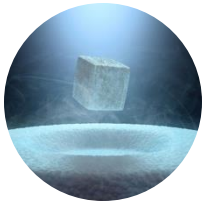
Machine Intelligence:: Deep Learning

Week 6: The DL Zoo and Generative AI

Beate Sick, Lilach Goren Huber, Pascal Bühler

Institut für Datenanalyse und Prozessdesign
Zürcher Hochschule für Angewandte Wissenschaften

About me



PhD theoretical physics, Weizmann Institute, Israel
Modelling high temperature superconducting materials.



Physicist + Data Scientist, Pentalum Ltd., Israel
Algorithms for LiDAR-based wind measurements.



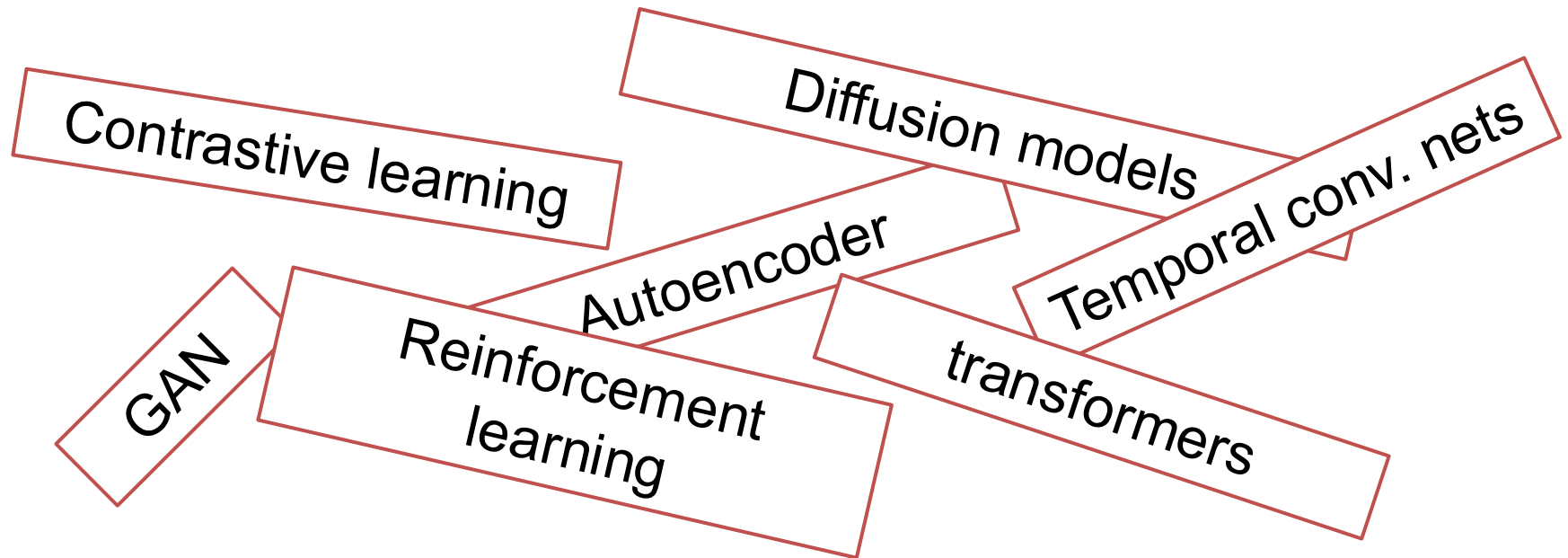
Since 2013: Senior Scientist & Lecturer, IDP, ZHAW

- ✓ Machine Learning / Deep Learning
- ✓ Industrial machines and infrastructure systems.
- ✓ Anomaly detection, fault diagnostics and predictive maintenance.

Lilach.gorenhuber@zhaw.ch

Goal of today

- Until now: you learned how to design your own DL models (architecture, activation, loss, tricks).
- There are many more ingredients/ concepts that one can combine together.

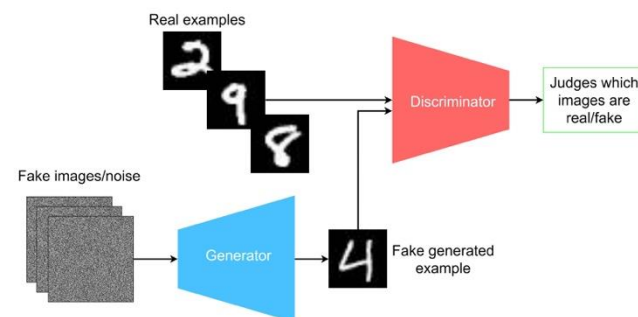
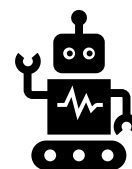
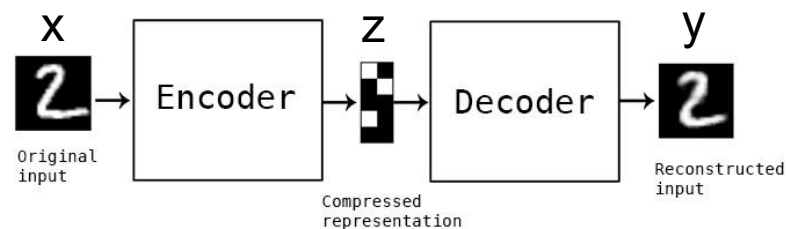
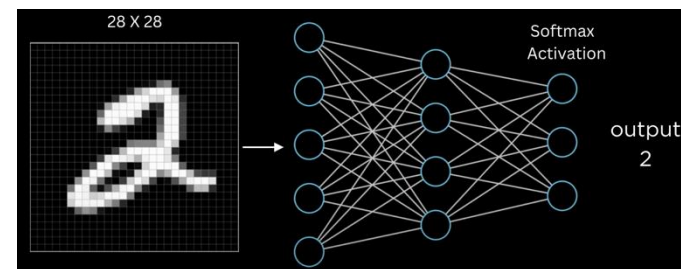


To search for the right possibilities you should use the correct lingo.

- ➔ Understand your task type.
- ➔ Have a general idea of the type of models that are out there.

Model Tasks of Deep Learning

- **Prediction:** Learns $p(y|x)$ - output based on input (classification, regression, forecasting, token prediction)
- **Representation Learning:** Learns latent embeddings or features that preserve important properties of data.
- **Decision-making:** Selects actions in an environment to maximize cumulative reward (Reinforcement Learning agents)
- **Generation:** Learns $p(x)$ or $p(x|z)$ - creates data (images, text, etc.)



Until now we focused on prediction tasks* and representation learning.

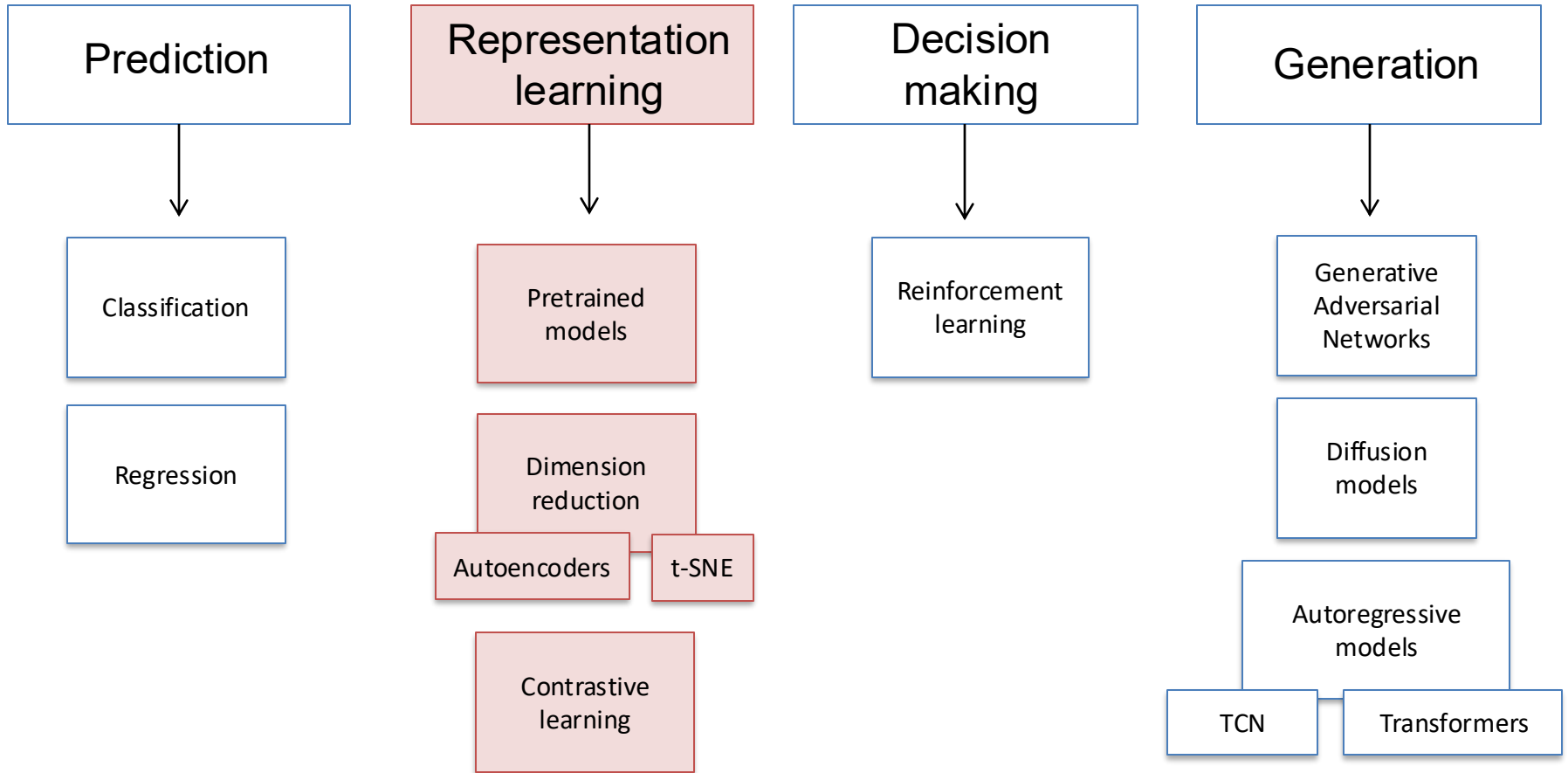
* Also to make decisions...

Today: lots of name-dropping, not much details.

<https://medium.com/@koushikkushal95/>

<https://blog.keras.io/building-autoencoders-in-keras.html>

Order in the zoo?



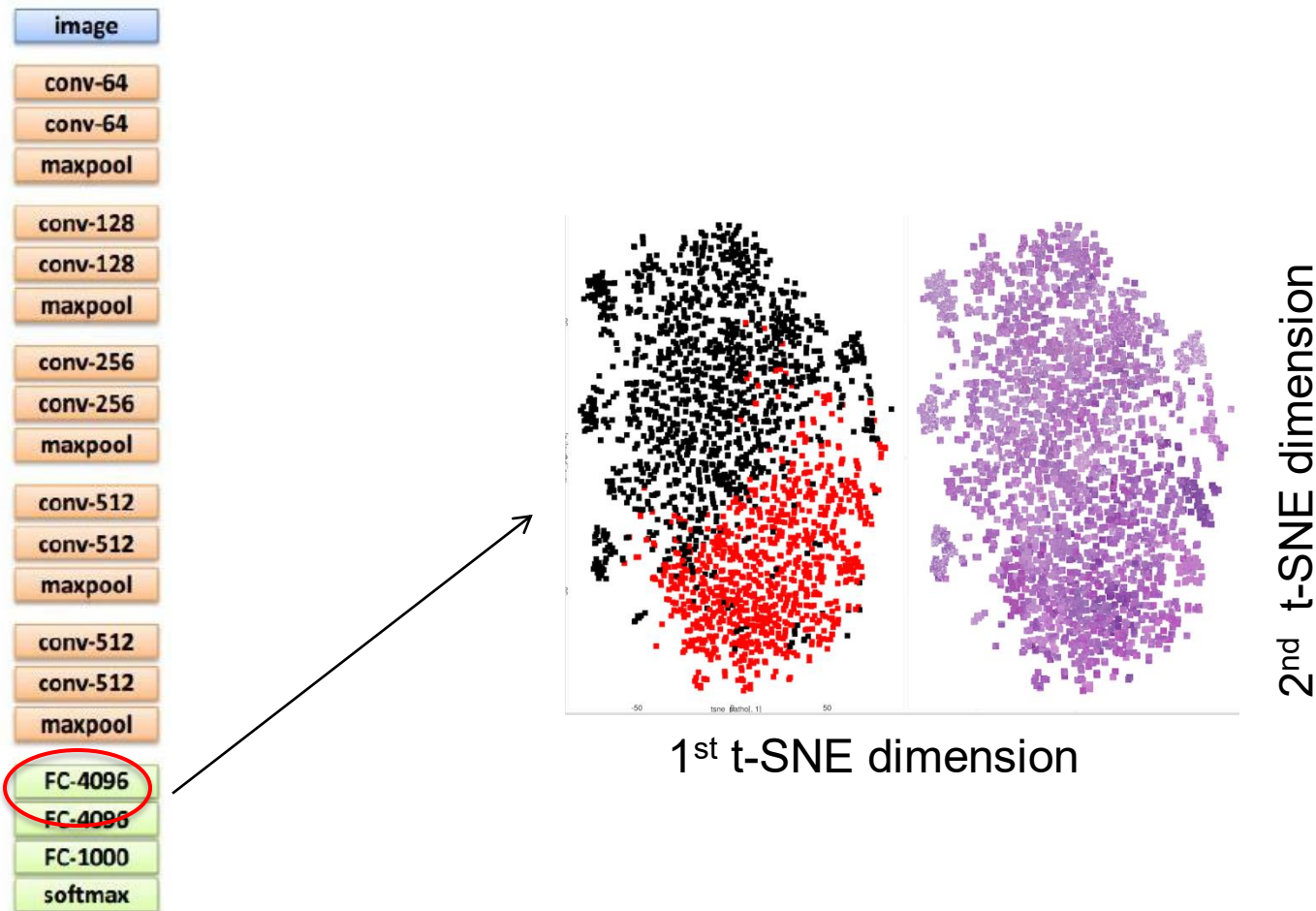
Representation learning

- **Learn latent embeddings** or features that preserve important properties of data.
- For a downstream task e.g classification, clustering, visualization, anomaly detection...
- We saw it with pretrained CNNs (ResNet, VGG...).
- We saw it in a supervised setup.
- Can also be **unsupervised** or **self-supervised** → trained without labels.
- Examples:
 - Dimension reduction: (PCA, t-SNE, UMAP,) Autoencoder NNs.
 - Contrastive learning.



t-SNE (t-Distributed Stochastic Neighbor Embedding)

Visualize high-dimensional data in 2D or 3D by placing similar points close together and dissimilar points far apart.

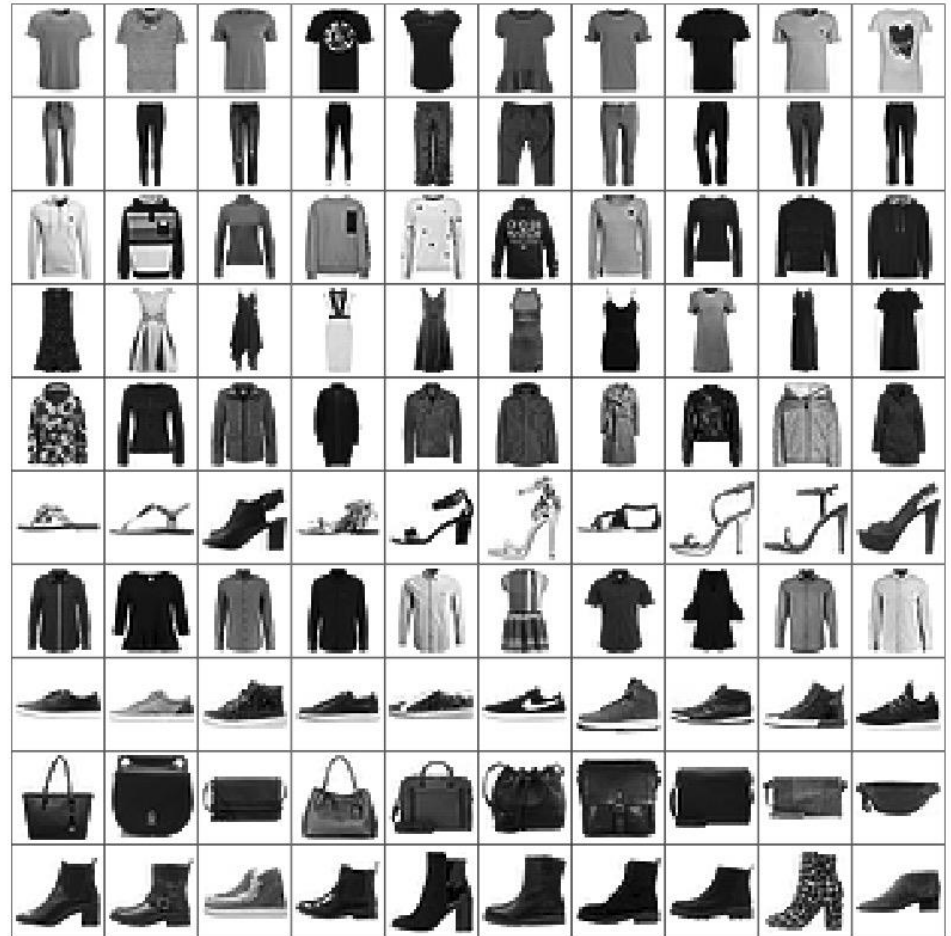


<https://www.youtube.com/watch?v=U-s8q6HshZw>

Example: t-SNE to visualize fashion article classes

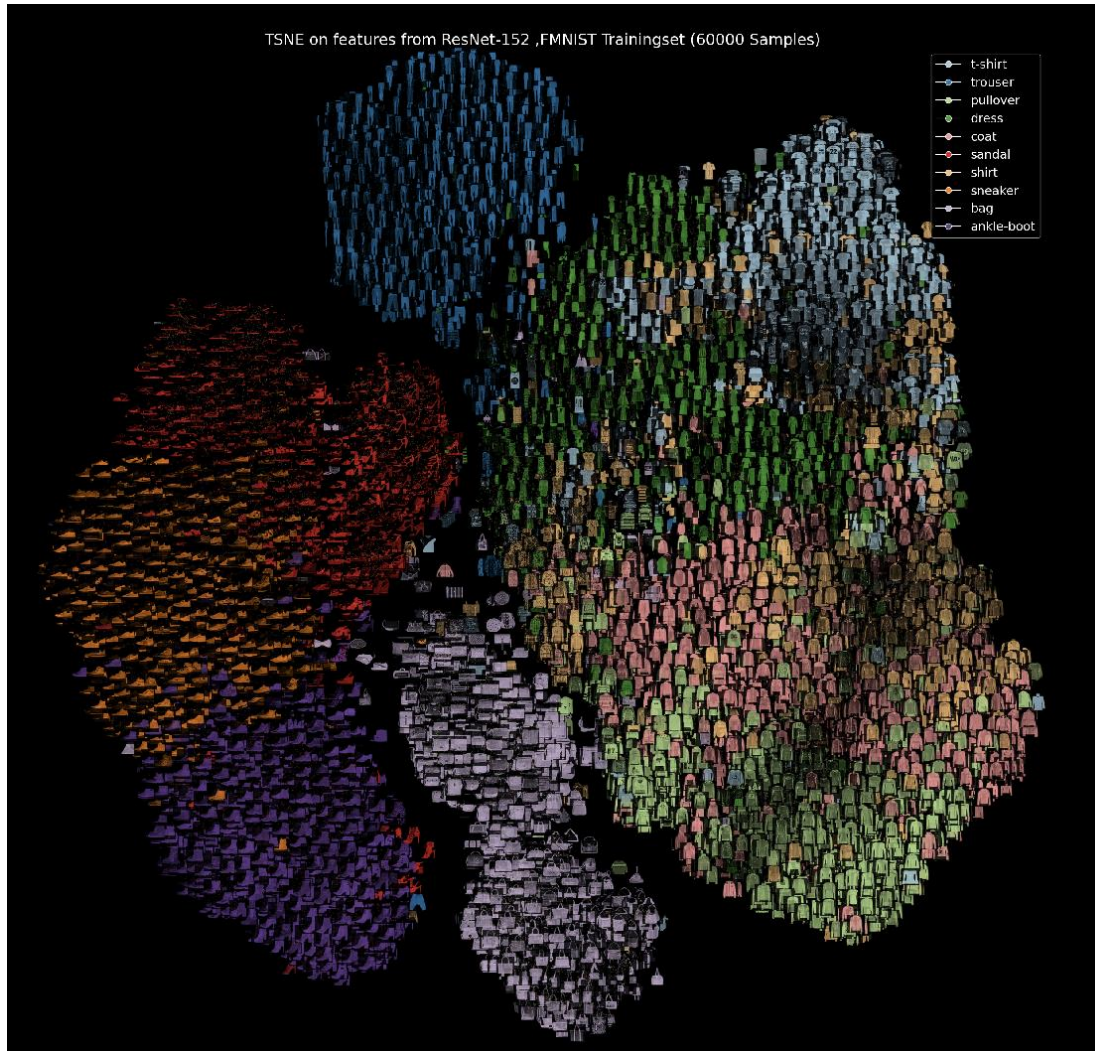
Fashion-MNIST Dataset

- Created by Zalando
- 10 article classes
- Typically used for classification benchmarking.
- Each image has $28 \times 28 = 784$ pixels.



t-SNE on FMNIST features

t-SNE on ResNet-152 features
(FMNIST training set 60'000 Samples)



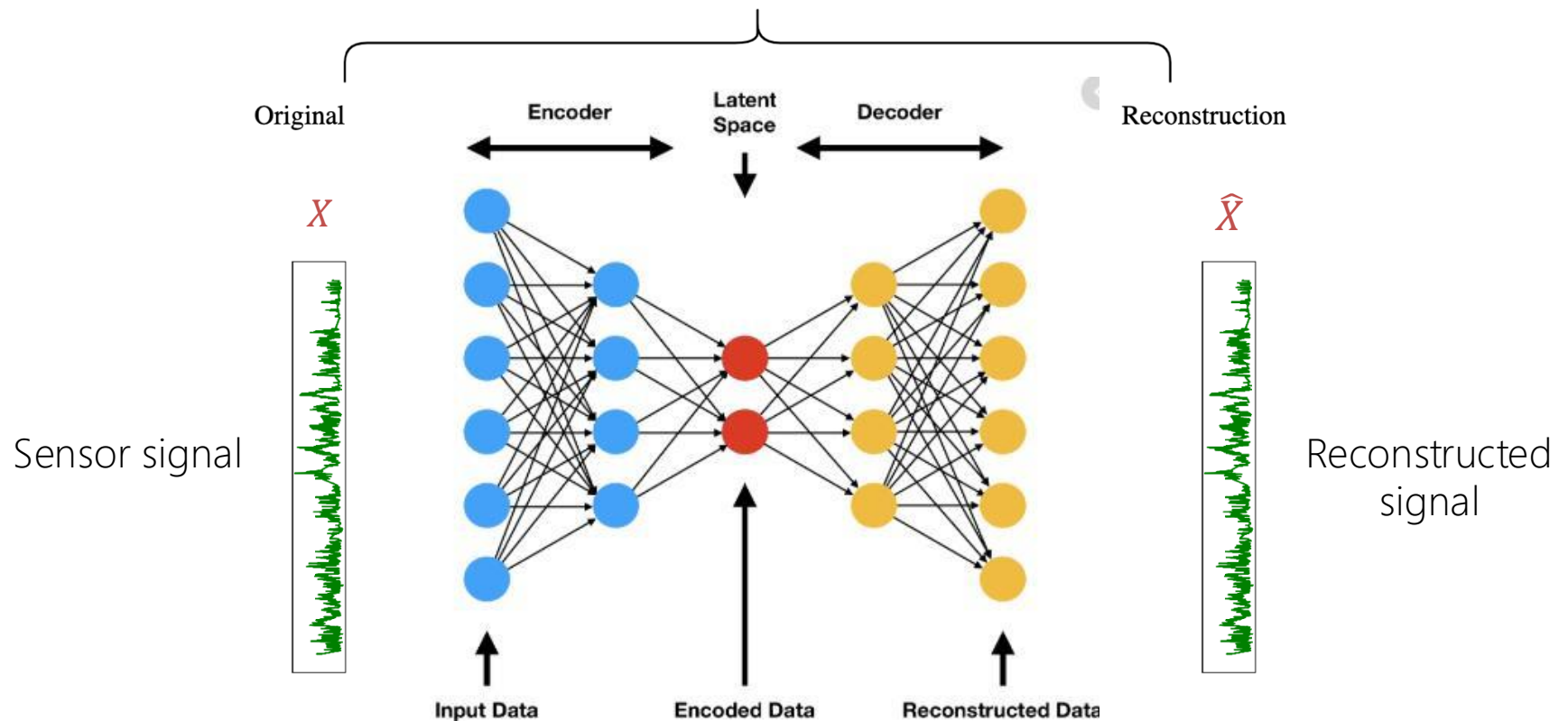
- Use ResNet to extract 2048 features from each image.
 - Input dim = 2048
 - Reduce to dim=2 to visualize.
 - Use the article shape as symbol.
- ➔ Trousers and bags are easily separated.
- ➔ T-shirts and dresses are close to each other.

Dimension reduction & reconstruction for anomaly detection

Autoencoder NN: train NN to reconstruct your data.

$$Loss = MSE = \frac{1}{n} \sum_{i=1}^n \|X_i - \hat{X}_i\|^2$$

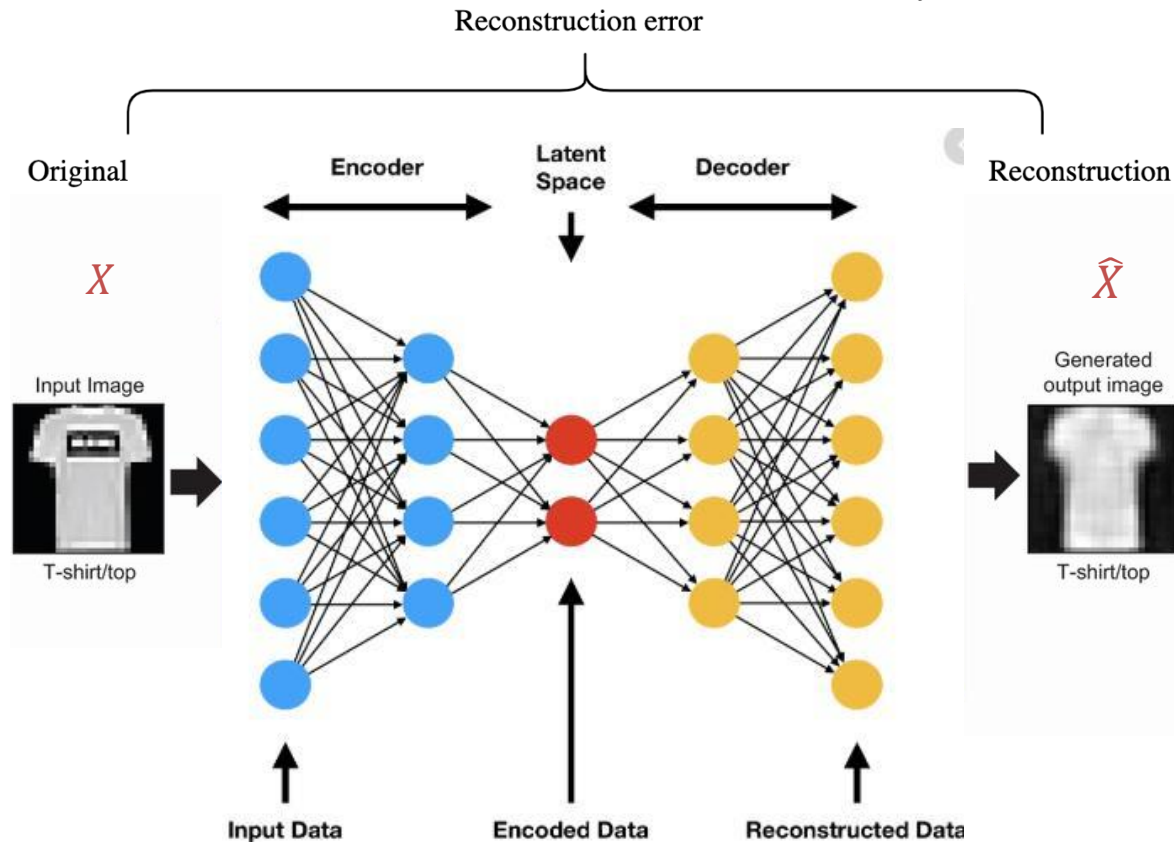
Reconstruction error



Dimension reduction & reconstruction for anomaly detection

Autoencoder NN

$$Loss = MSE = \frac{1}{n} \sum_{i=1}^n \|X_i - \hat{X}_i\|^2$$



Dimension reduction & reconstruction for anomaly detection

Now we can do the same with conv. layers:

$$Loss = MSE = \frac{1}{n} \sum_{i=1}^n \|X_i - \hat{X}_i\|^2$$

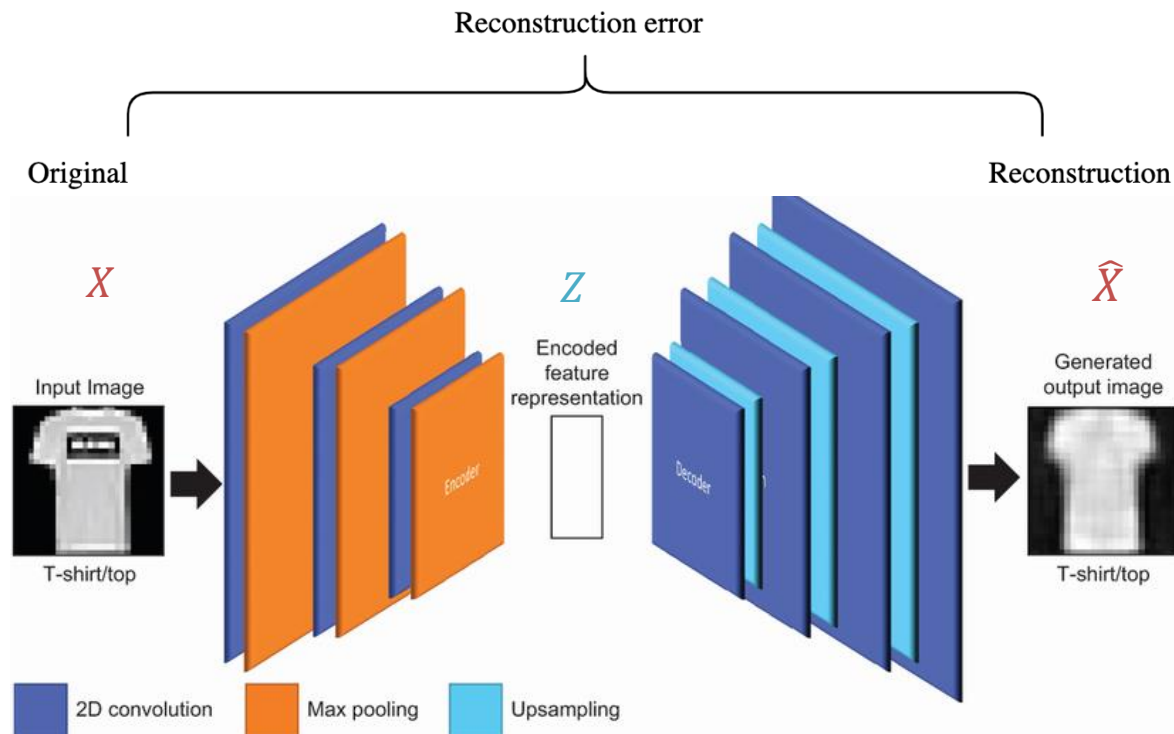
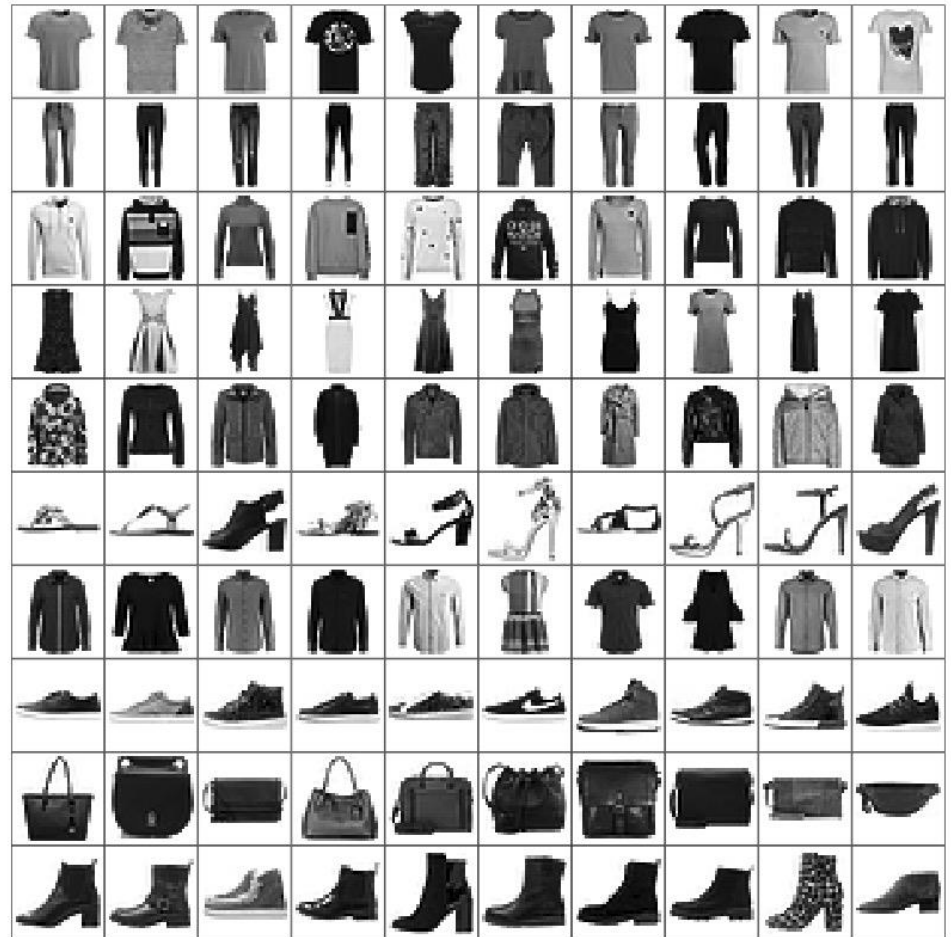


Image source: evolutionary-deep-learning

Example: image anomaly detection

Fashion-MNIST Dataset

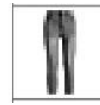
- Created by Zalando
- 10 article classes
- Typically used for classification benchmarking.
- Can be used for image anomaly detection benchmarking.



Adapt F-MNIST for anomaly detection

- One vs. All:

- <trousers> is the majority = normal.
- A mix of all other articles is the minority = abnormal.

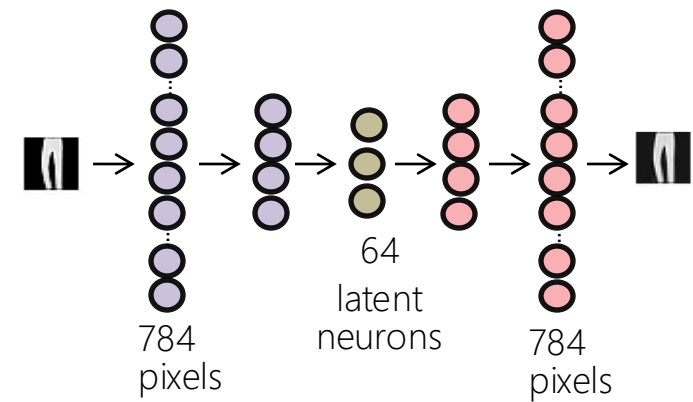


- Training: use images of the normal class to train an autoencoder.
- Inference: other articles will be badly reconstructed

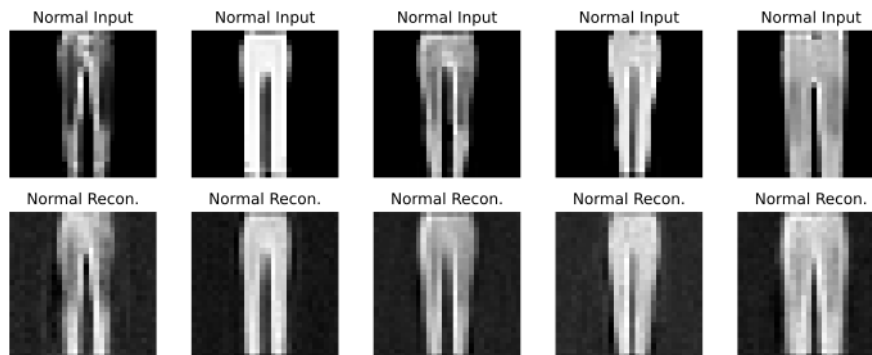
➔ Use recon. errors as anomaly scores.

Image Anomaly Detection with an Autoencoder NN

Example: Fully connected Autoencoder NNs
Trousers vs. all



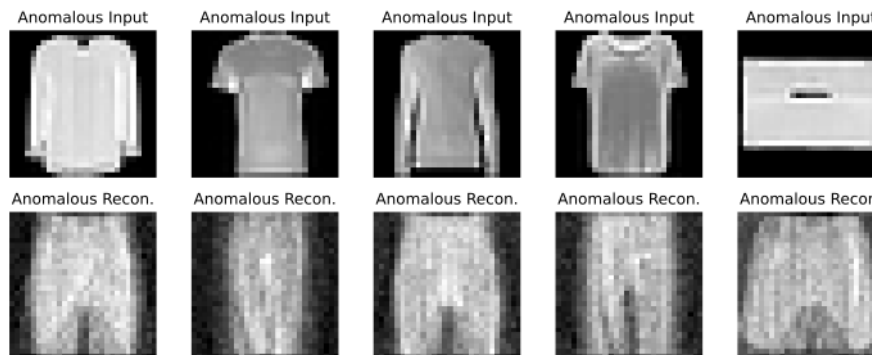
Normal
images



input

reconstruction

Abnormal
images



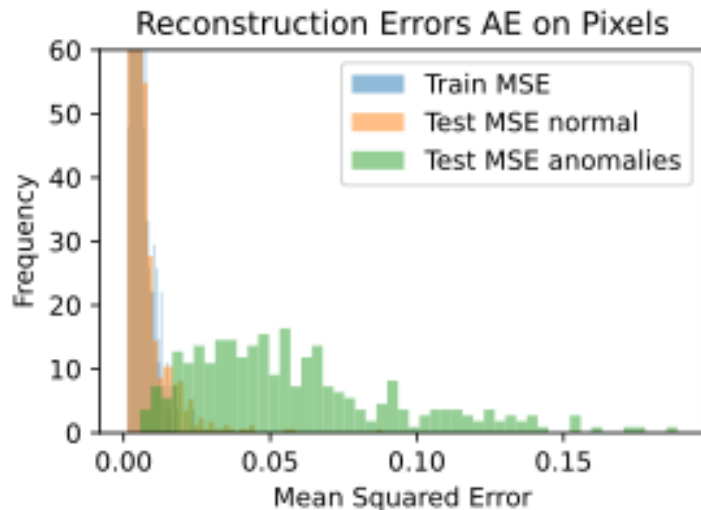
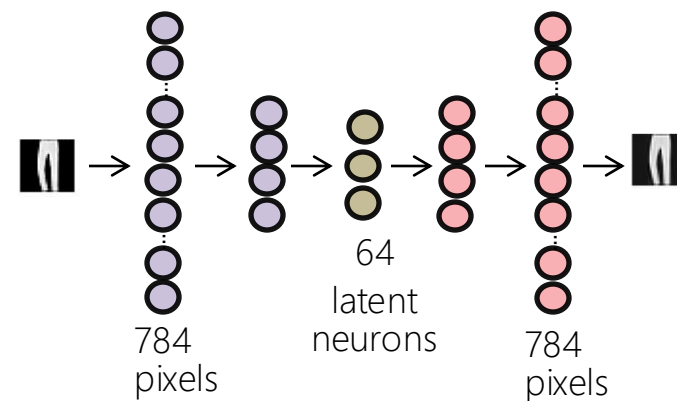
input

reconstruction

Image Anomaly Detection with an Autoencoder NN

- Example: trousers vs. all
- Calculate the mean squared reconstruction error (MSE) for each image.

$$MSE = \frac{1}{N} \sum_{i=1}^{N=784} \|X_i - \hat{X}_i\|^2$$



- ➔ Task: anomaly detection.
- ➔ Solution: Unsupervised* representation learning.

*not quite, but it is a longer story...

[Example on colab](#)

Dimension reduction: Autoencoder is a non linear version of PCA

PCA reminder:

- Linear dimensionality reduction from dimension p to dimension $L < p$.
- Intuition:
 - Small axes $\rightarrow$ variable does not explain much $\rightarrow$ less important $\rightarrow$ drop it.

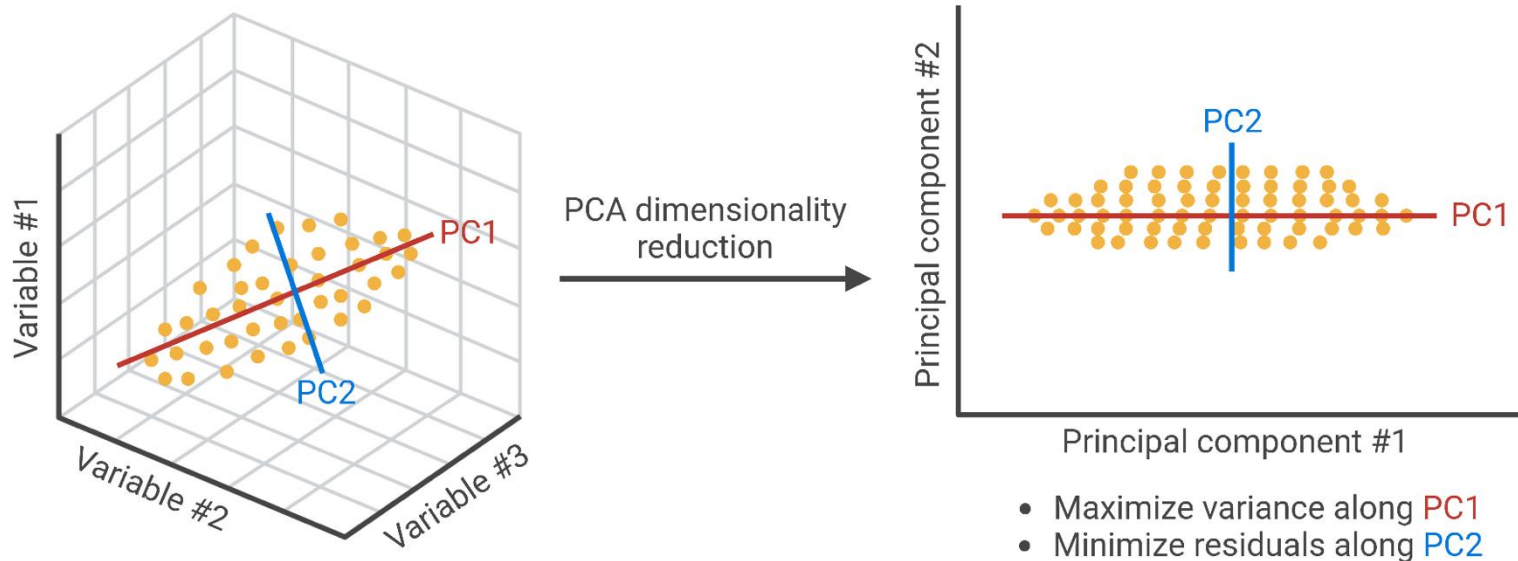


Image source: <https://www.biorender.com/template/principal-component-analysis-pca-transformation>

Dimension reduction & reconstruction for anomaly detection

1. Reduce the dimension of the data to a lower dimensional space.
2. Reconstruct the data from the reduced space.
3. Use only normal data to fix the PCA transformation.

- ➔ Reconstructing abnormal samples is unsuccessful, assuming they are generated by a different process.
- ➔ Reconstruction errors can be used to detect anomalies.

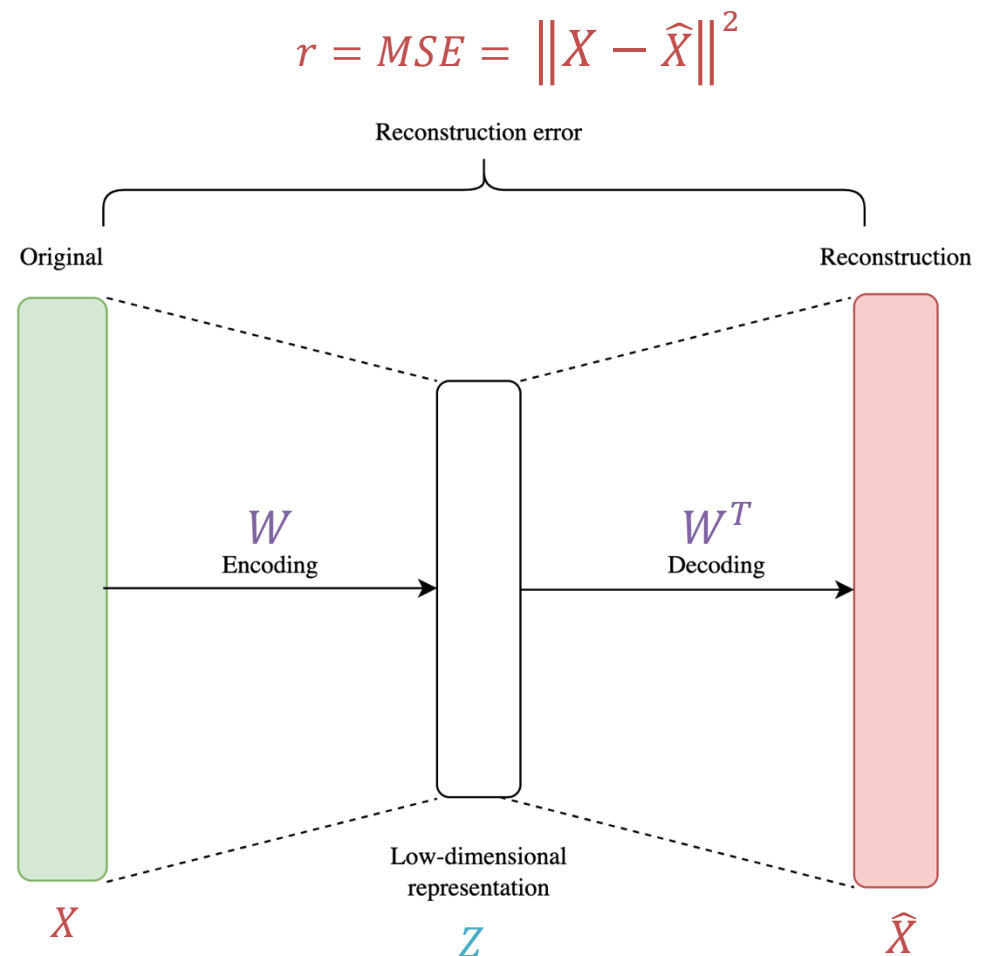


Image source: <https://ymohamedahmed.github.io/post/pca/>

Representation learning with contrastive loss

Learn useful data representations by comparing

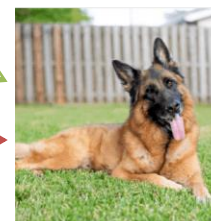
similar vs. **dissimilar** pairs.

How? Choose the right loss.

- **Positive pairs: same object, different views**
- **Negative pairs: different objects**



Positive: I^+



Anchor: I^a



Negative: I^-

Goal:

Pull **positive** pairs closer, push **negative** pairs apart in embedding space.

➔ Enables powerful self-supervised learning without needing labels.

Classification



Detection



Segmentation



Contrastive learning

Start with an **anchor** sample

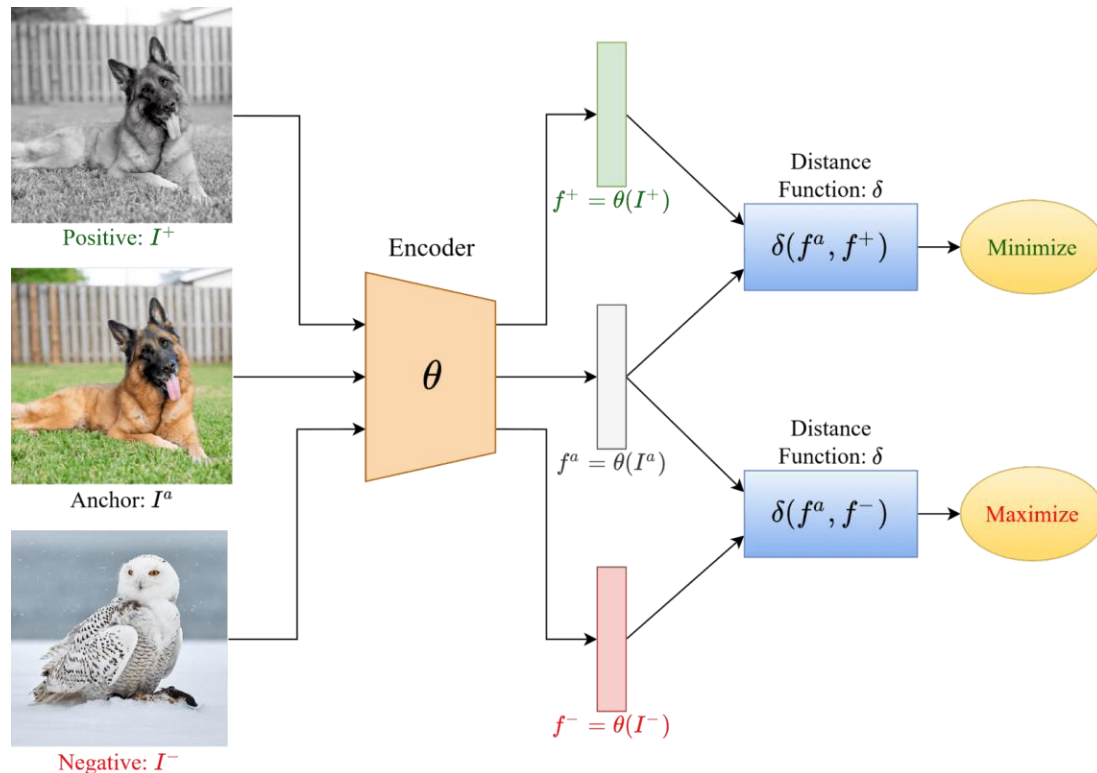
- E.g., an image of a dog

Create augmentations (views)

- Two slightly different versions of the same image → **positive** pair.
- The rest are **negative**.

Feed through an encoder NN

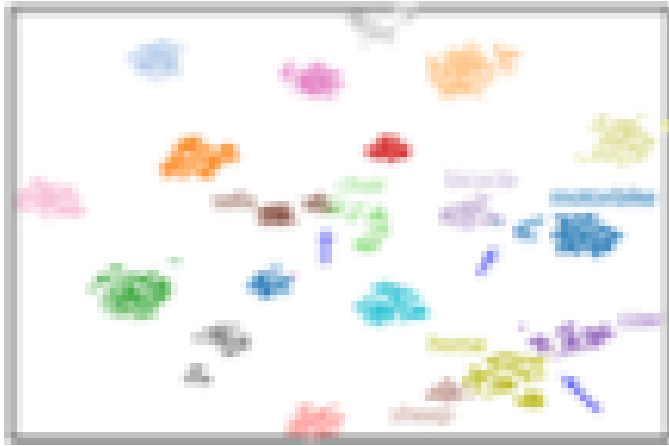
- Train with Contrastive Loss
- Learn representations (vectors)



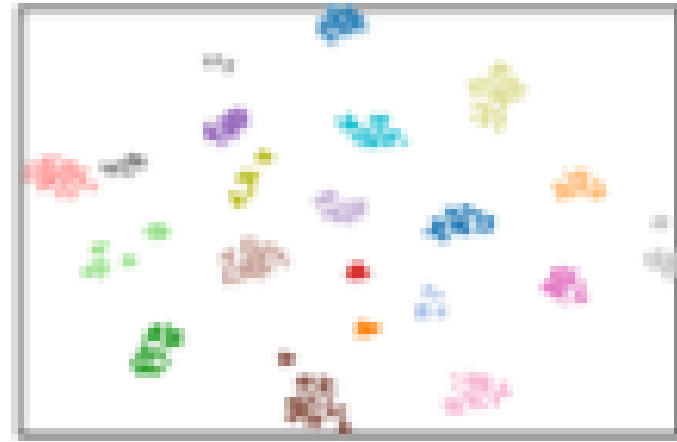
$$\text{Loss} = -\log \frac{\exp(\text{sim}(f^a, f^+)/\tau)}{\sum_k \exp(\text{sim}(f^a, f^k)/\tau)}$$

Similarity function e.g. $f^a \cdot f^k$

Example: object detection with contrastive learning (CL)

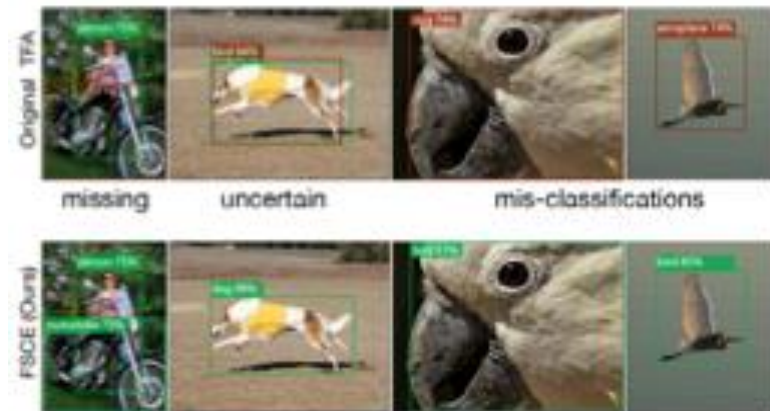


No contrastive loss



With contrastive loss

- CL pushes **similar** images closer together, and **dissimilar** images further away.

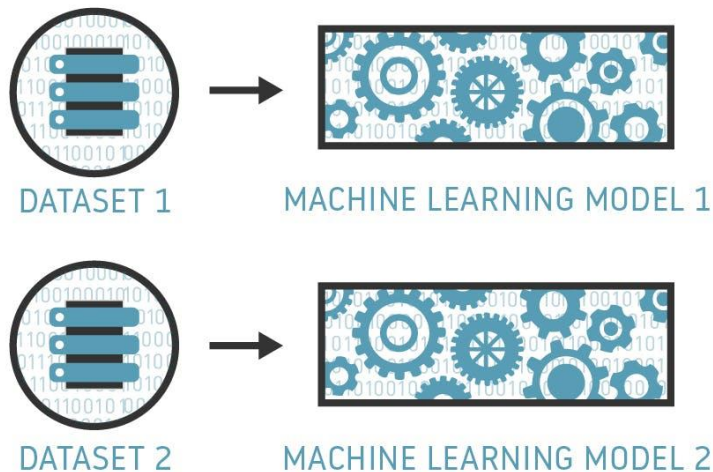


(c) bad cases rescued by our FSCE

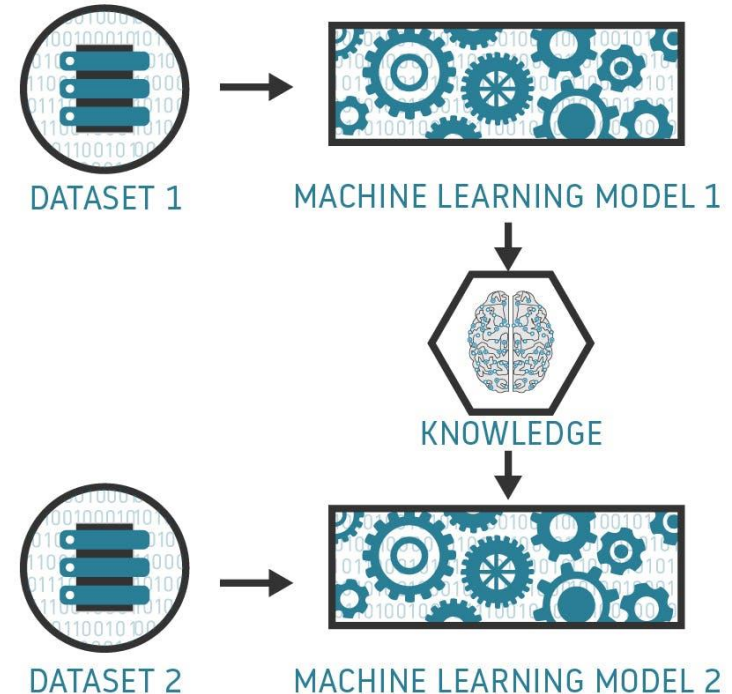
Contrastive learning can improve transfer learning

- Capture meaningful features from source dataset
- Use them to predict on target set (classify, regress, detect...)
- **Few-shot/ zero-shot** learning if few/no labels for target set (chatGPT)

TRADITIONAL MACHINE LEARNING



TRANSFER LEARNING



Benefits of contrastive learning

Label-Free Learning

Learns from raw data using clever structure (augmentations).

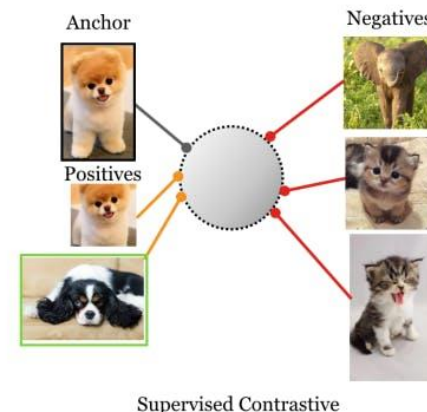
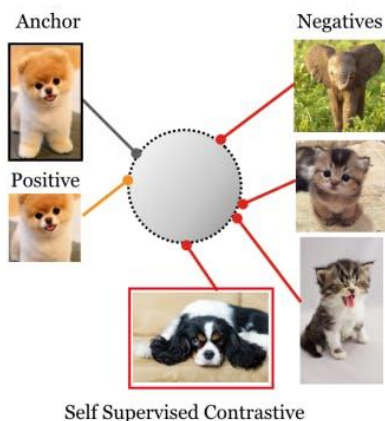
Strong Representations

effective for downstream tasks like classification, retrieval, clustering.

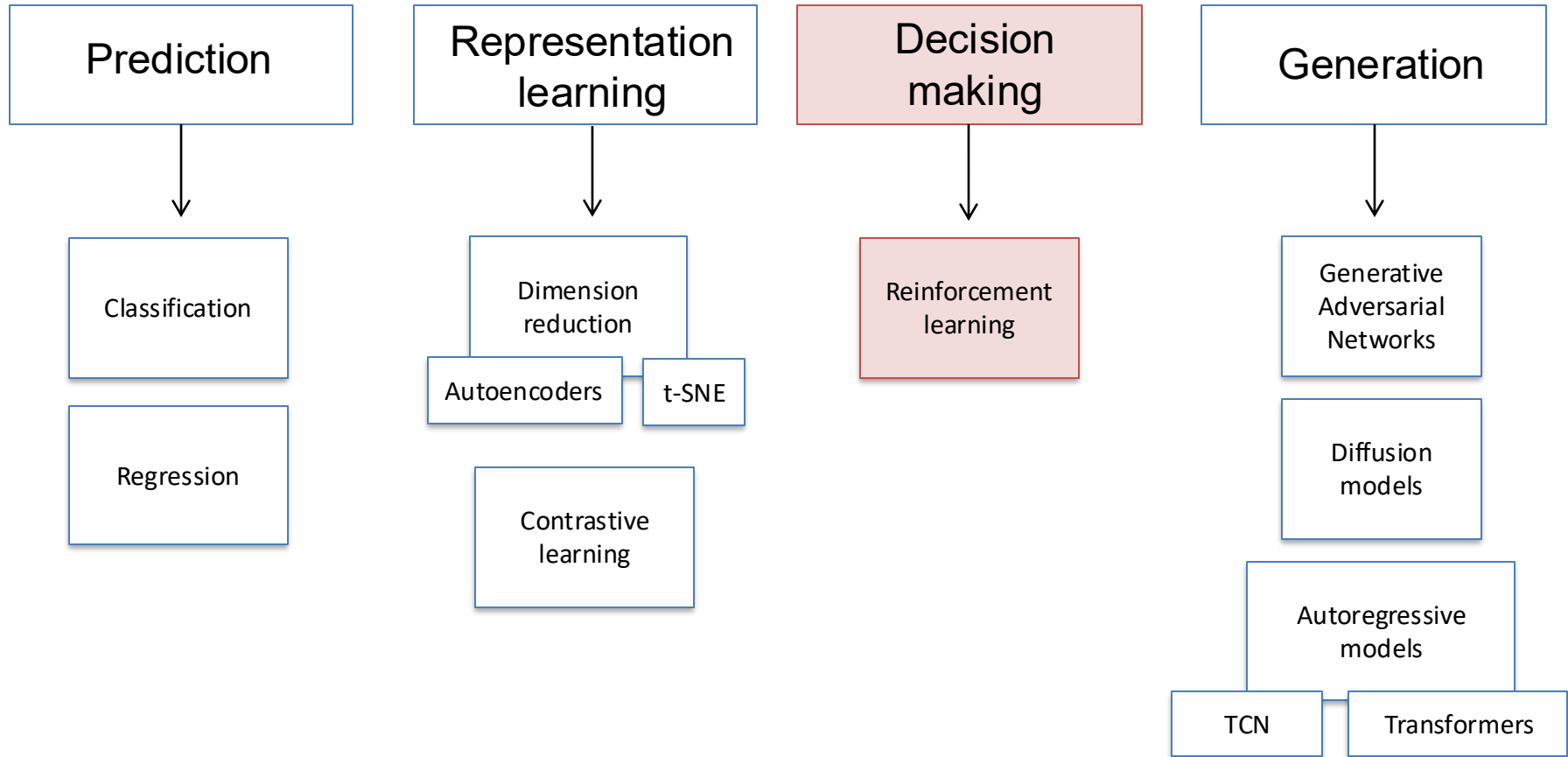
Can be used in supervised classification and regression settings

Building Block for:

- SimCLR, MoCo → image models
- CLIP → text-image pairs
- BYOL → no negative pairs



Order in the zoo?



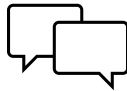
Reinforcement learning applications

Games



Mastered Chess, Go, Atari games without being told the rules.

Language Models



Fine-tuning GPT-4 or Claude to align more with human preferences

Robotics



Robots learning complex tasks like parkour, manipulation without millions of real-world trials.

Recommendation Systems



Netflix, YouTube, TikTok use variations of RL to optimize content suggestions without real-time trial and error.

Healthcare



Used for treatment planning (e.g., optimal sepsis treatment policies learned from ICU data).

Reinforcement learning (RL)

- Task: optimize decision making.
- RL: learn to make good decisions through trial and error.

➔ The goal: select actions to **maximize long-term reward**.

Expected return starting from state s , taking action a :

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[R_t | s_t = s, a_t = a]$$

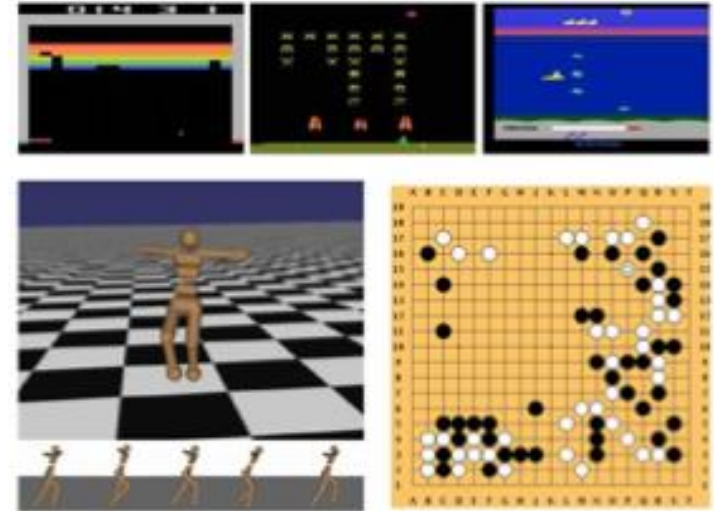
State agent sees

action agent picks

Total discounted reward:

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

reward agent gets



Deep Reinforcement Learning

Train a neural network to approximate and maximize the reward:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[R_t | s_t = s, a_t = a]$$

	Predictive Models	Reinforcement Learning
Loss Function	Supervised loss (e.g., MSE, cross-entropy)	Reward-driven loss (e.g., temporal difference error, policy gradient)
Feedback	From known labels	From interaction rewards

[AlphaGo by Google's DeepMind:](#)

Some RL resources for Self Study

Deep RL Bootcamp (Berkeley)

- <https://sites.google.com/view/deep-rl-bootcamp/home>

Book: R. Sutton and A. Barto, “Reinforcement Learning: An Introduction”

- THE book on RL, free online: <http://incompleteideas.net/book/the-book.html>

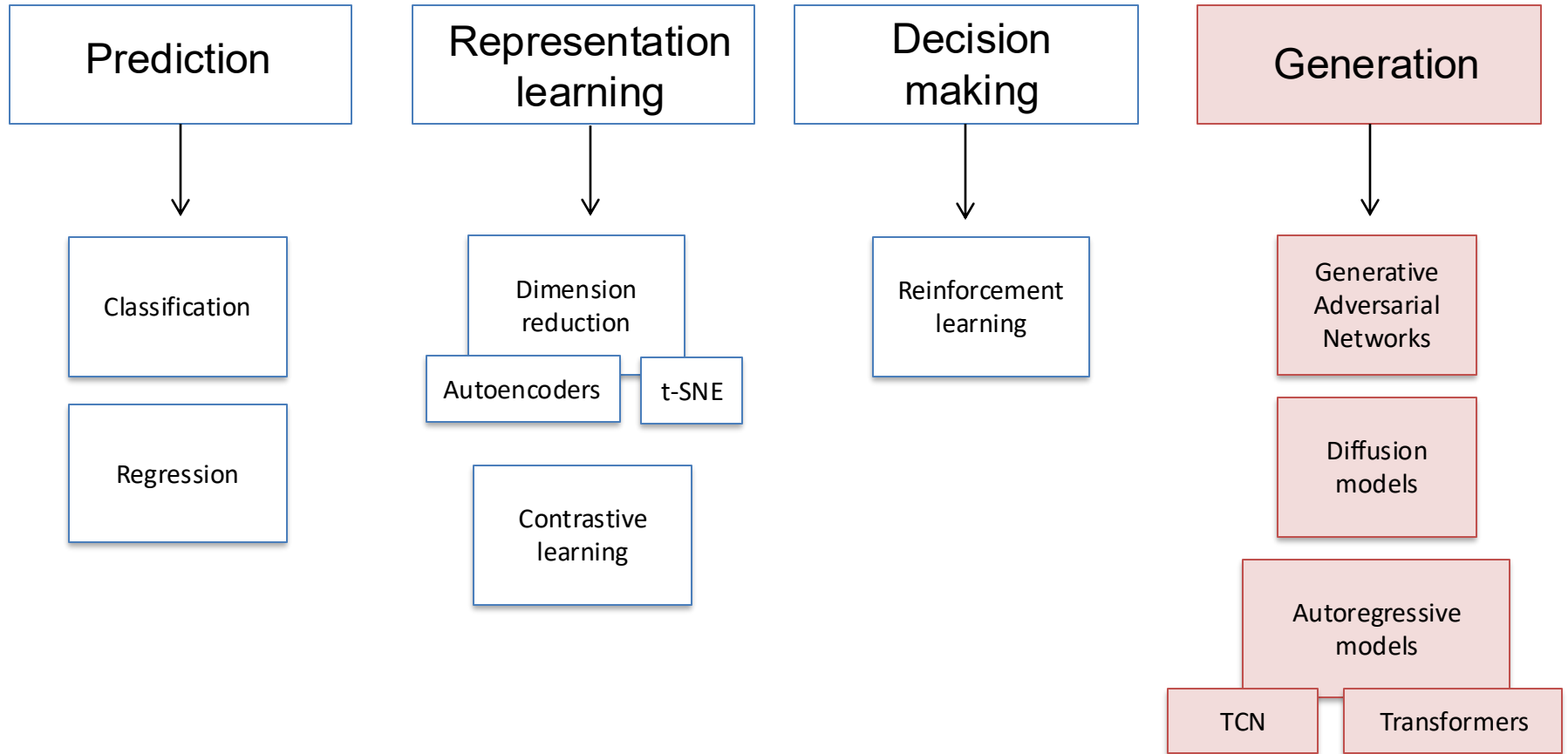
Other online courses

- Berkeley full DRL course <http://rail.eecs.berkeley.edu/deeprlcourse/>
- OpenAI: <http://spinningup.openai.com/en/latest/index.html>
- Lectures by David Silver (DeepMind):
<http://www0.cs.ucl.ac.uk/staff/d.silver/web/Teaching.html>

Blog posts

- A. Karpathy on Policy Gradient methods
<http://karpathy.github.io/2016/05/31/rl/>
- A. Irpan: DRL doesn't work yet <https://www.alexirpan.com/2018/02/14/rl-hard.html>
- Lessons learned from reproducing a DRL paper
<http://amid.fish/reproducing-deep-rl>
- Lilian Weng (Lil'Log) <https://lilianweng.github.io/lil-log/>

Order in the zoo?



Generative models

Learn $p(x)$ or $p(x|z)$ — create data (images, text, video, sound.)

Generative AI

Generative
Adversarial
Networks

Diffusion
models

Autoregressive
models

Variational
Auto-
Encoders

Flow-
based
models

Energy-
based
models

DEEFAKE

Usually CNN
backbone
(Deepfake)



Image from
text
generation
(Dall-E,
Midjourney,
Sora)



Temporal Conv.
Networks
(timeseries
forecasting)
Transformers
(ChatGPT, Siri,
Alexa, sora)



Avatar
generation
from
pictures



Song
creation
(Jukebox),
fashion
image
generation
(Zalando)



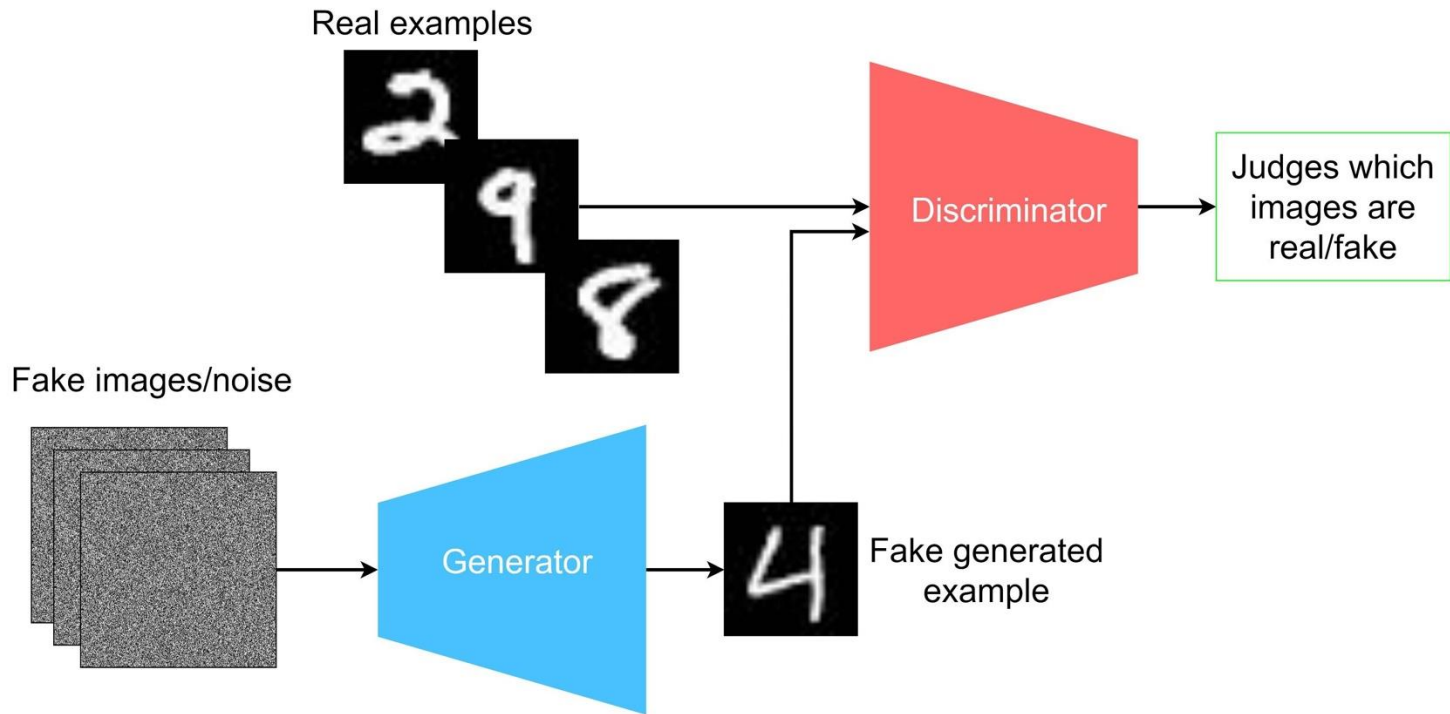
Image
denoising
(photoshop)

Generative Adversarial Networks (GANs)

<https://thispersondoesnotexist.com/>

GANs are a type of neural network architecture designed to generate new data that is similar to the input data they were trained on.

Creator: Ian Goodfellow (2014).



GANs example

When training begins, the generator produces obviously fake data, and the discriminator quickly learns to tell that it's fake:



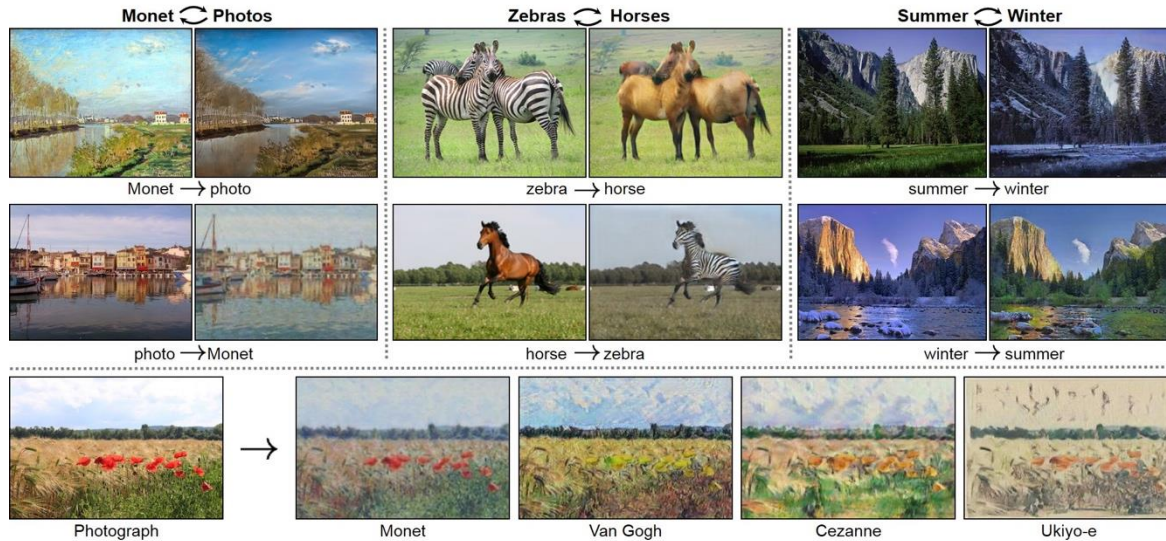
As training progresses, the generator gets closer to producing output that can fool the discriminator:



Finally, if generator training goes well, the discriminator gets worse at telling the difference between real and fake. It starts to classify fake data as real, and its accuracy decreases.



Popular GAN examples



CycleGAN: Image-to-image translation
(<https://junyanz.github.io/CycleGAN/> 2017)

Loss function ensures cyclic consistency → No need for paired data

StyleGAN (Nvidia 2018):
Introduce the input into various generator layers → learn different aspects of input separately.

→ e.g change face and not hair.



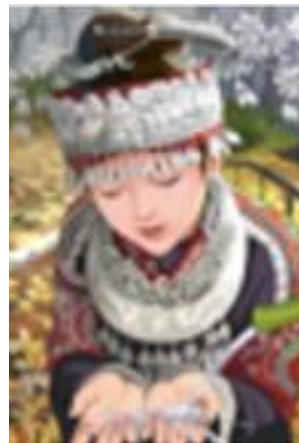
Popular GAN examples

Super Resolution GAN:
enhancing low-res images.
(Twitter: Ledig et al 2017).

original



blurred



Restored with GAN



Deepfake

https://www.youtube.com/channel/UCKpH0CKltc73e4wh0_pgL3g

<https://www.youtube.com/watch?v=cQ54GDm1eL0>

GANs Pros and Cons

Pros:

- Can generate realistic, high-quality data
- Useful in data augmentation
- Capable of unsupervised learning (no need for labeled data)

Cons:

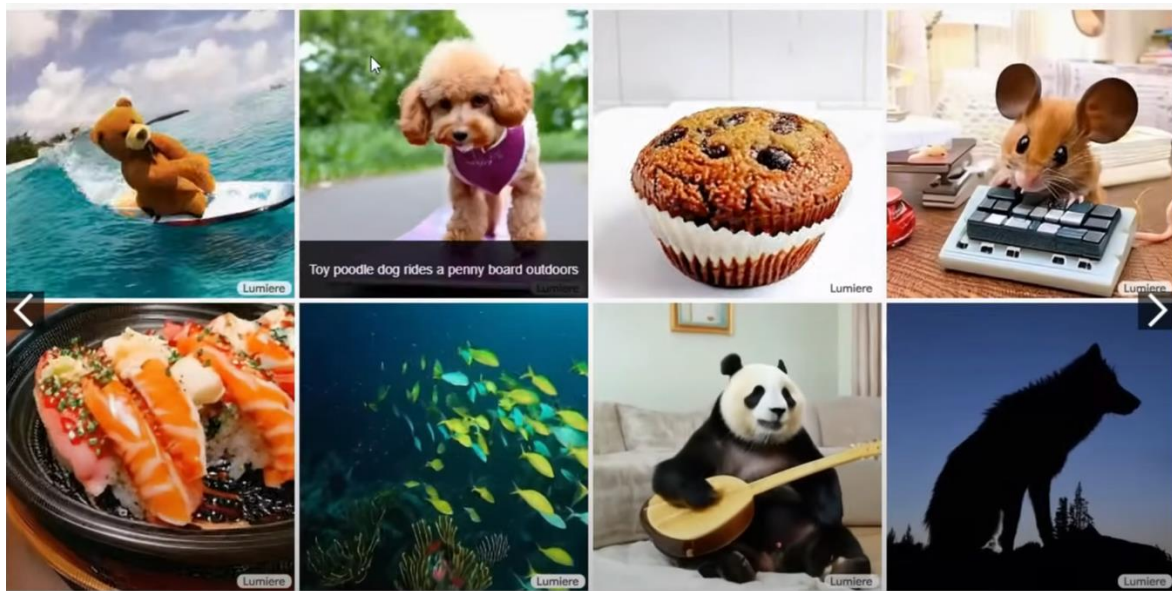
- Training instability: GANs are hard to train
- Mode collapse: generator might produce limited diversity
- Visual artefacts: melting textures, ghosts of previous images...
- No inherent probabilistic interpretation: how likely is an image?

Diffusion Models

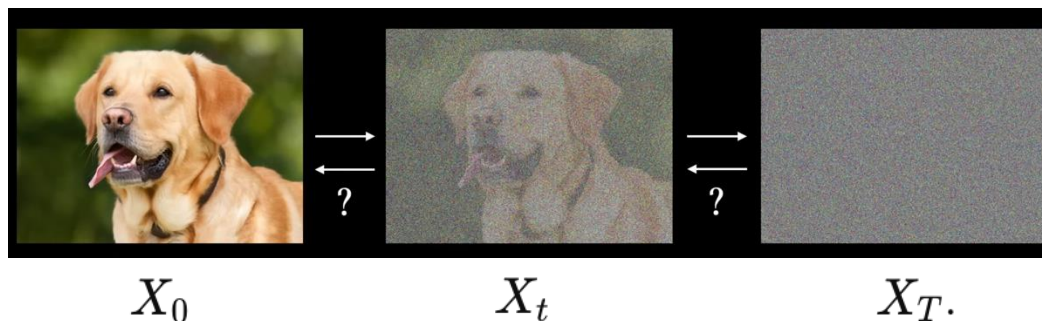
Sora Prompt: Reflections in the window of a train traveling through the Tokyo suburbs.

<https://www.youtube.com/watch?v=6eKud22jMlo&t=4s>

”Denoising Diffusion Probabilistic Models” Ho, Jain, Abbeel 2020.



How does it (more or less) work?



- Start with a real image X_0
- Diffuse it gradually with noise over many steps (usually 100-1000)
- Add the noise following a fixed rule: $X_t = \sqrt{\bar{\alpha}_t}X_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$

$\swarrow$
Fixed noise schedule

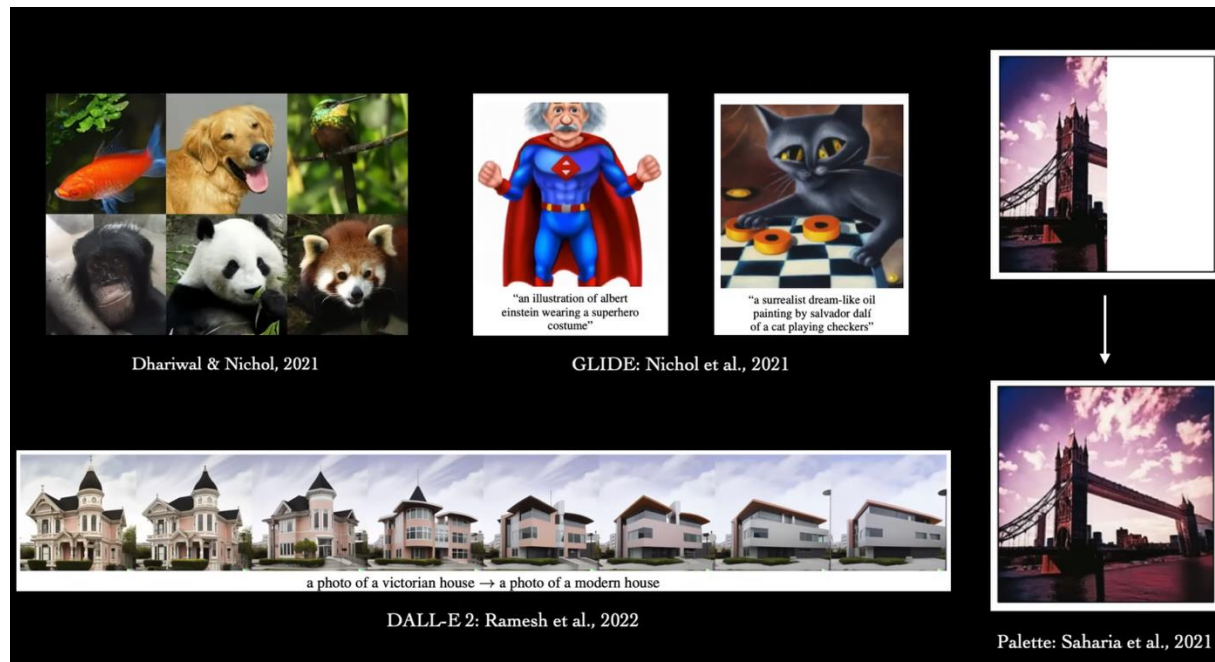
$\downarrow$
True noise added $\epsilon \sim \mathcal{N}(0, I)$
- Train the model to undo the noise: $L = \mathbb{E}_{X_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(X_t, t)\|^2]$

$\downarrow$
The model tries to predict the noise added at each step

→ GANs generate images from scratch, DMs predict and remove small amounts of noise → stable, accurate and diverse generation

Applications of Diffusion Models

- **Text-to-image generation** (*DALL-E2, Stable Diffusion, Midjourney, Imagen*)
- **Text-to-video generation** (*Sora, Pika Labs*)
- **Image inpainting**: Filling missing parts of an image
- **Image editing**: Modifying specific objects or styles
- **Super-resolution**: Enhancing image quality
- **Audio & speech generation** (*WhisperDifussion, AudioML*)
- **Scientific use**: Protein folding, drug design (*OpenFold*)



Diffusion Models Pros and Cons

Pros:

- High-quality and diverse output. Outperforms GANs in image quality.
- Stable training, unlike GANs
- Works well with conditioning (e.g., text, masks)
- Probabilistic interpretation → can estimate the likelihood for a specific output.

Cons:

- Slow sampling: multiple denoising steps.
- Resource-intensive (though improving).

Where GANs Still Shine

1. Real-Time Applications / Low Latency Inference

- Diffusion models need hundreds of sampling steps. GANs produce outputs in a single forward pass.
- Use cases:
 - Virtual backgrounds in Zoom
 - Augmented reality face filters
 - Super-Resolution on-device pics enhancement (phones, embedded systems).



2. Image-to-Image Translation (Pix2Pix, CycleGAN, GauGAN...)

- These tasks need precise structure-to-image mapping. Diffusion versions are developed, but GANs are still faster and more plug-and-play.
- Use cases:
 - Turning sketches into photos (e.g. GauGAN)
 - Day-to-night, season translation
 - Medical imaging (CT $\leftrightarrow$ MRI)



3. Latent space editing

- StyleGAN's latent space is interpretable and structured. Latent editing (age, smile, pose, etc.) is easier in GANs than in diffusion.
- Use cases:
 - Face editing apps
 - Fashion design tweaks
 - Identity blending (e.g., morph two people's faces)



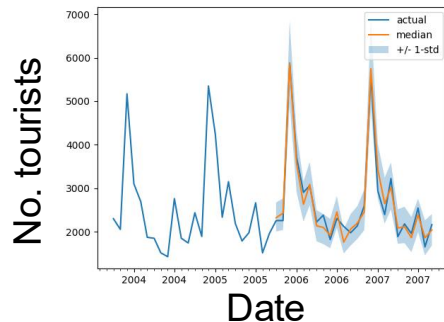
Autoregressive Generative Models

- The idea:
 - Generate data one step at a time.
 - Each new output is conditioned on the previous outputs:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

- Applications: time series, language, audio.

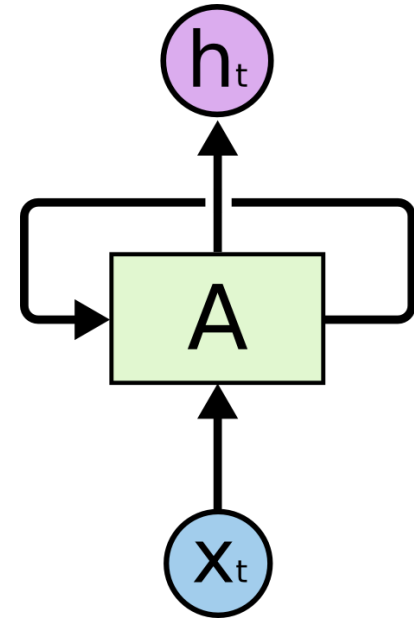
<https://google-research.github.io/noise2music/>



- Model examples
 - Reminder: RNNs, LSTM.
 - Introduce Temporal Convolutions (TCN) as a variant.
 - Transformers.

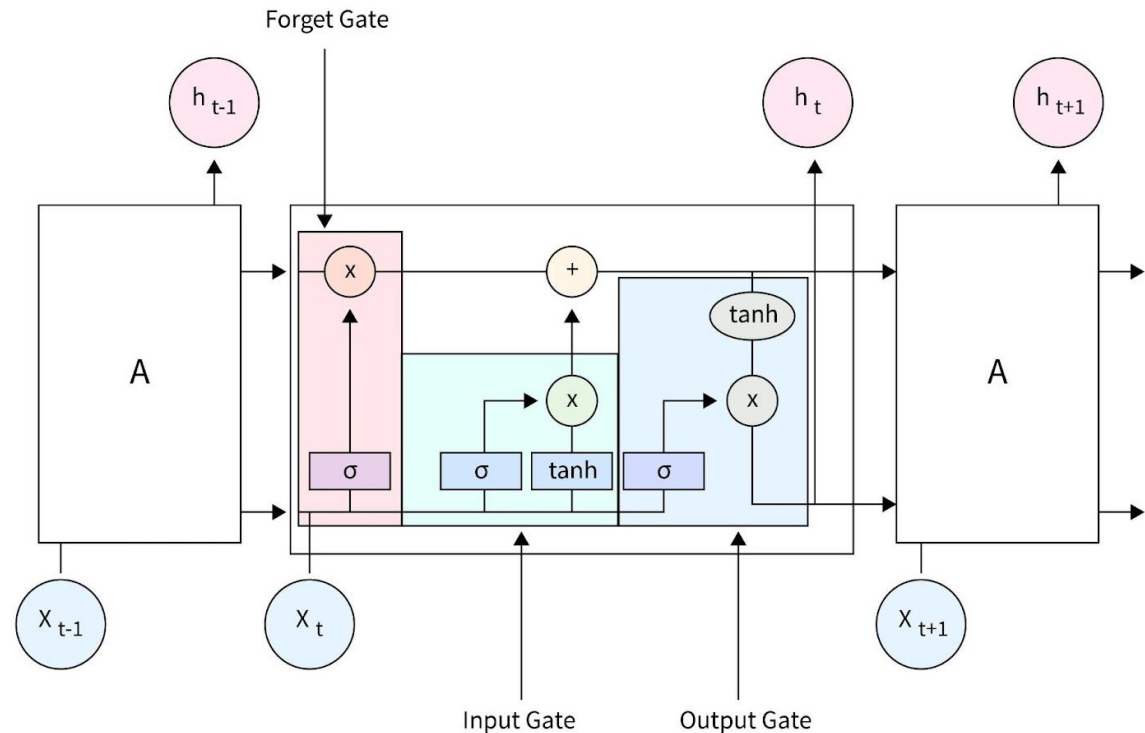
Reminder: Recursive NNs (RNNs)

- RNNs are core architectures that handle sequence data by maintaining memory across steps.
- Examples
 - Forecast the weather.
 - Predict stock or oil prices.
 - Predict machine condition (e.g engine health).
 - Predict the next word in a sentence.
- What is special?
 - Input size may change (e.g context length)
- Solution
 - Feed in a hidden state with all past information
 - Enforce the same weights recurrently across time.
- Limitation
 - Vanishing/exploding gradients due to propagation through time.



Reminder: Long Short Time Memory (LSTM) Networks

- Challenge: preserve also long term information.
- Solution:
 - By default keep information **unchanged through time** using a memory cell.
 - **Forget gate** and **input gate** slightly adjust it at each time step.



Temporal Convolution Networks (TCNs)

Alternative to LSTMs.

2016 Google's WaveNet was first:

- Deep generative model
- Produces raw audio waveforms
- Developed by google DeepMind 2016
- Sounds very natural compared to traditional text-to-speech (TTS)
(Traditional: Text → Linguistic analysis → Unit selection → Concatenation → Speech)
- Used in:
 - Google assistant
 - Google translate 

Temporal Convolution Networks (TCNs)

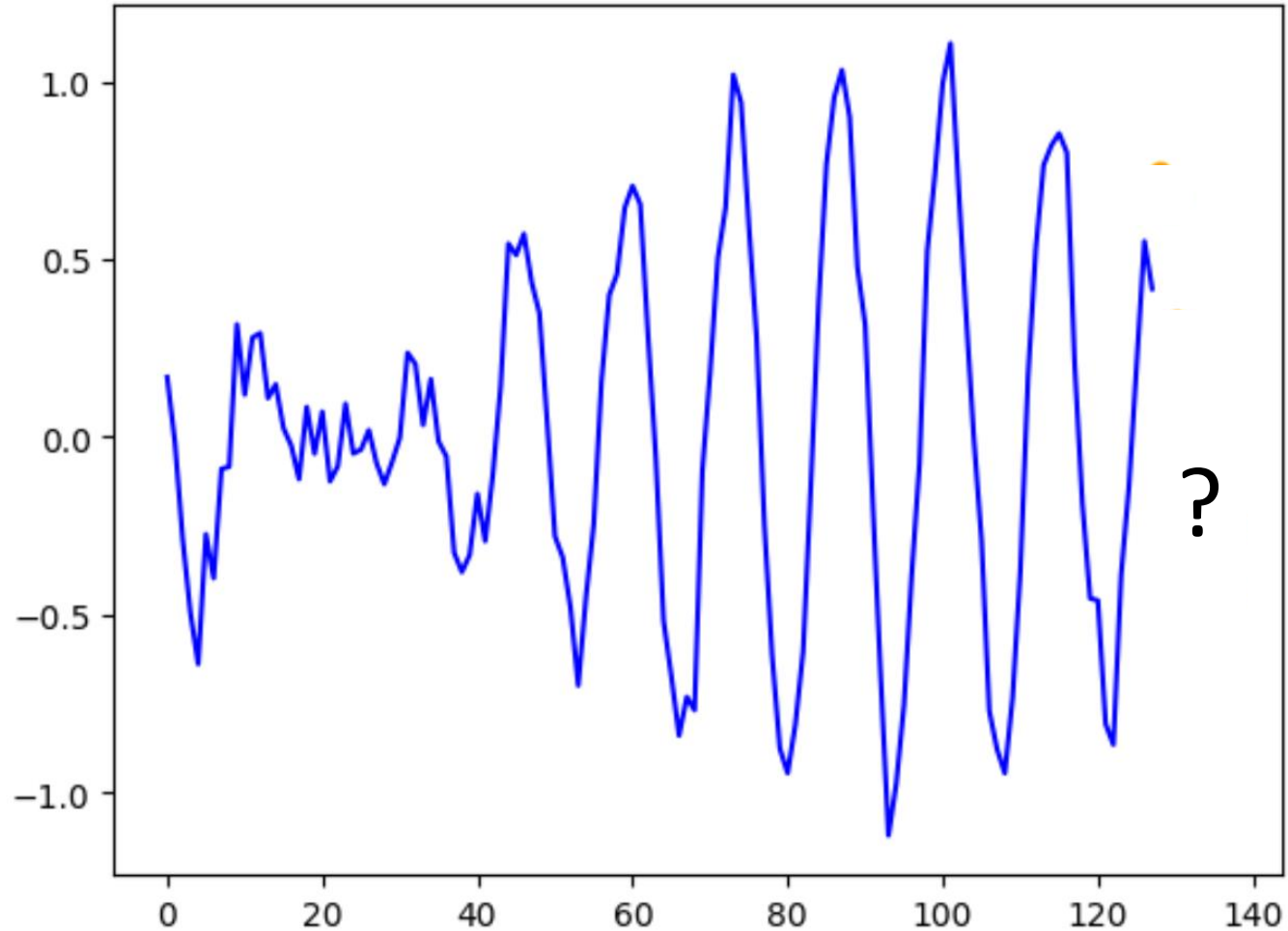
- 2016 Google's WaveNet
 1. 1D conv. layers
 2. Causal convolutions
 3. Dilation
 4. Residual connections
- Then TCN 2018, Bai et al.
 - Same core ideas
 - Flexible for different tasks (e.g time series, text)
 - No heavy post-processing
 - Easier to train



van den Oord, et al. (2016). *WaveNet: A generative model for raw audio*.

Bai et al (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling.

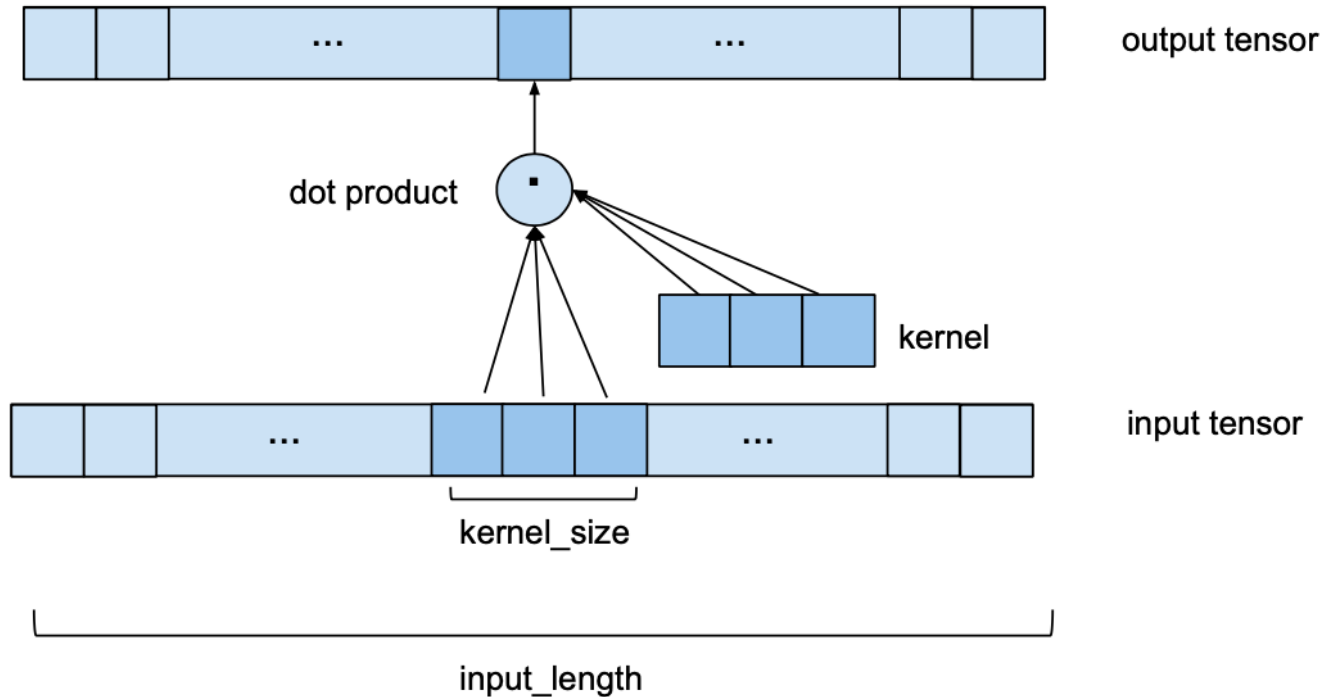
How to make forecasts based on a given time series?



1D convolutions

2Dconv: 3x3 kernels

1Dconv: 3x1 kernels

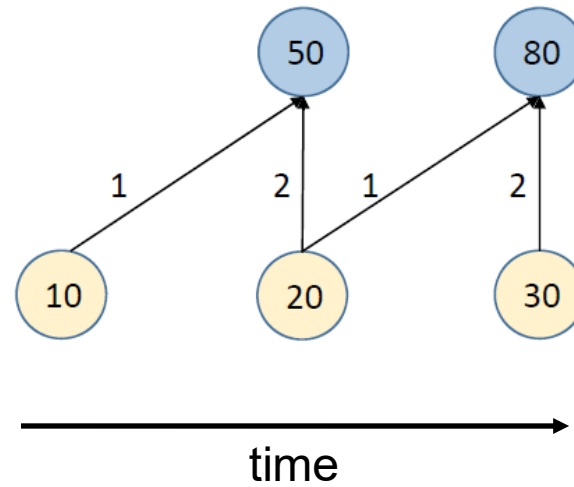


1D “causal” convolution for time-ordered data

Toy example:

Input X: 10,20,30

1D kernel of size 2: 1,2



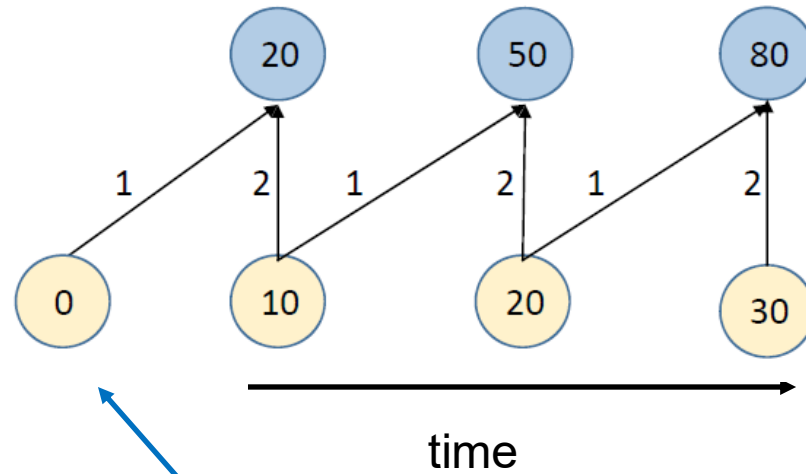
It's called “causal” networks, because the architecture ensured that only information from the past has an influence on the present and future.

Zero-padding in 1D “causal” CNNs

Toy example:

Input X: 10,20,30

1D kernel of size 2: 1,2

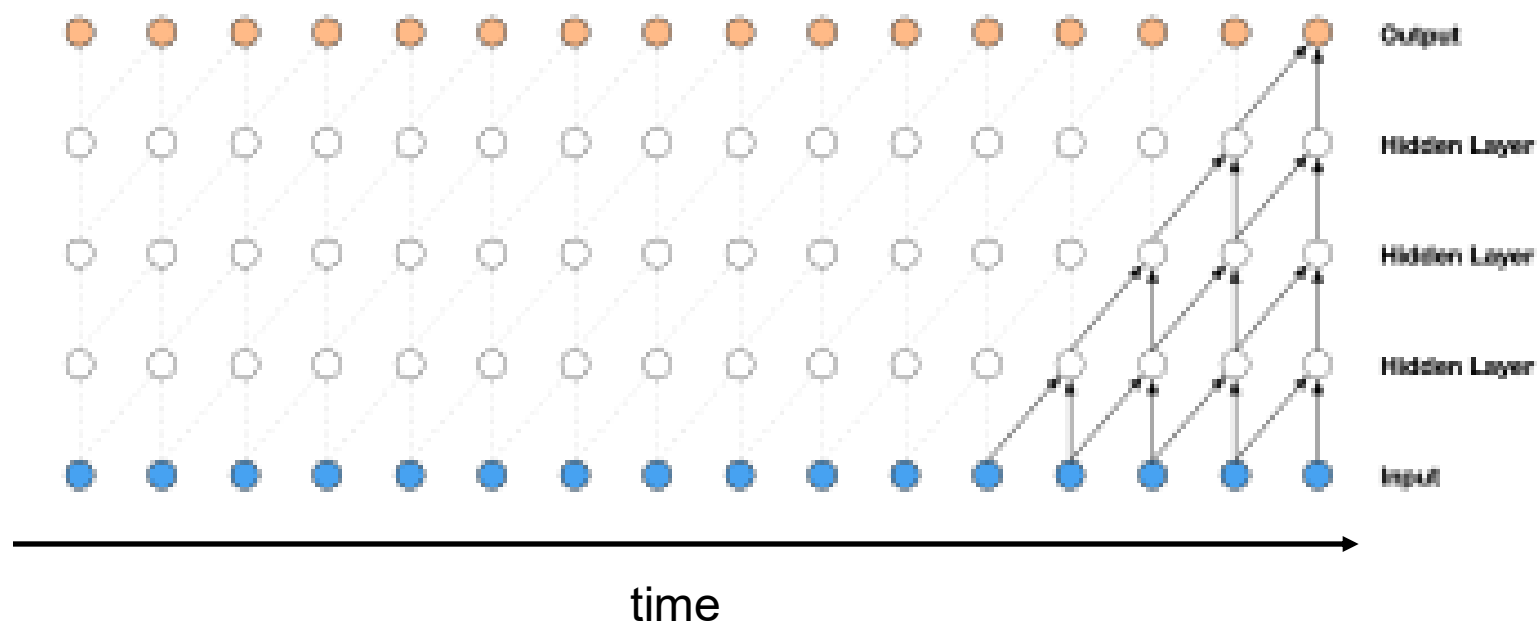


To make all layers the same size, a **zero padding** is added to the beginning of the input layers

1D “causal” convolution in Keras

```
model = Sequential()  
model.add(Convolution1D(filters=1,  
                        kernel_size=2,  
                        padding='causal',  
                        dilation_rate=1,  
                        use_bias=False,  
                        batch_input_shape=(None, 3, 1)))  
model.summary()
```

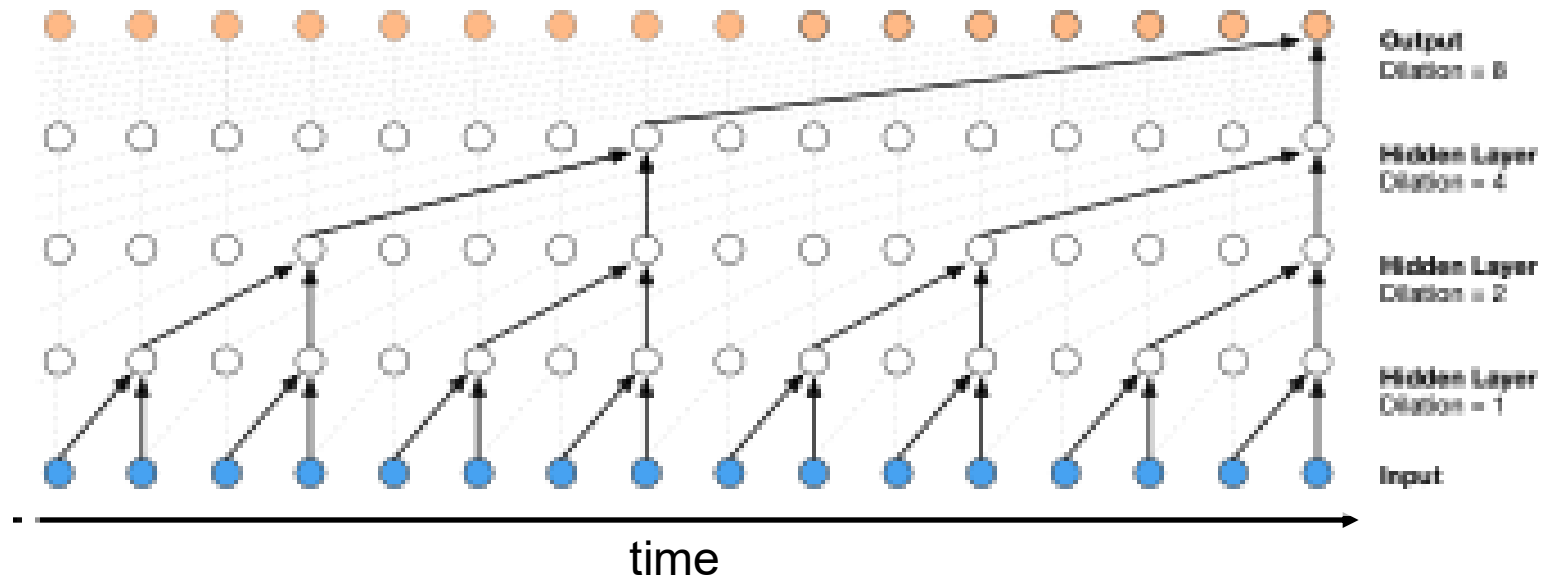
Stacking 1D “causal” convolutions



Stacking k causal 1D convolutions with kernel size 2 allows to look back k time-steps. After 4 layers each neuron has a “memory” of 4 time-steps back in the past.

1D conv. with Dilation to increase receptive field

To increase the context of neurons in the output layer, you can use “dilated” convolutions:



After 4 layers each neuron has a “context” of 15 time-steps back in the past.

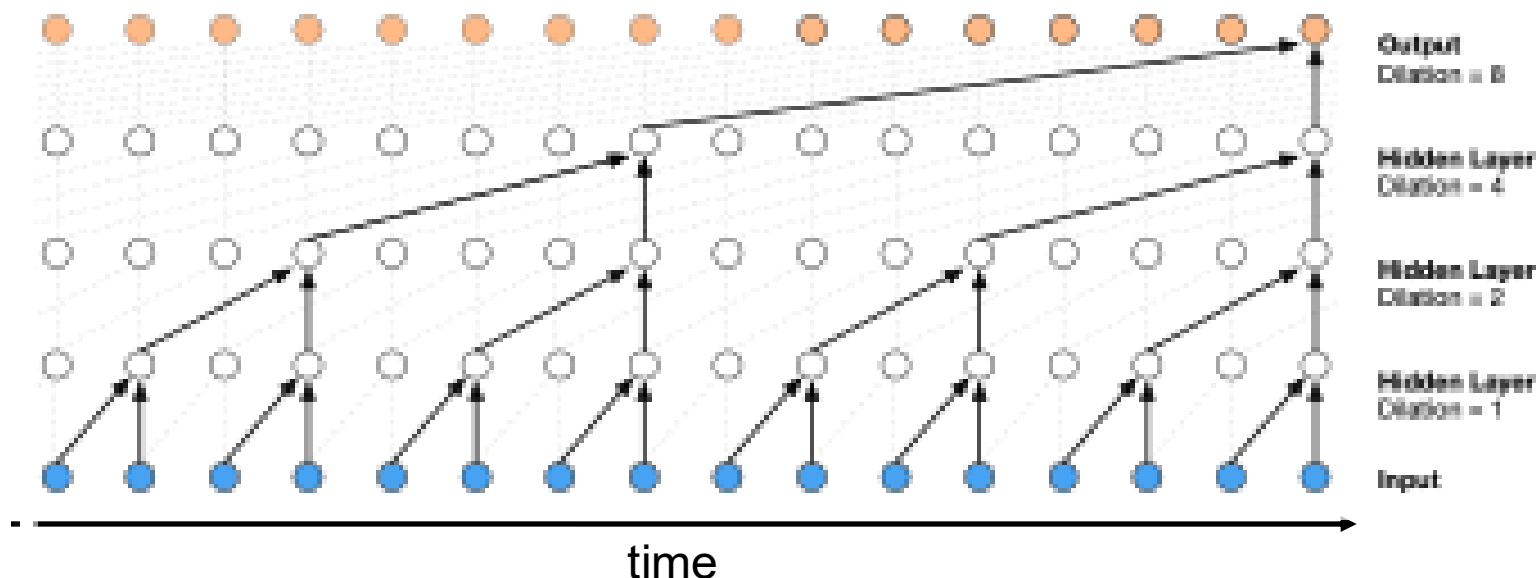
After 5 layers $2^5 - 1 = 31$ time steps.

Question: why not 1 filter of length 31?

In a (1,2,4,8,16) dilation, how many parameters do we save?

1D conv. with Dilation to increase receptive field

To increase the context of neurons in the output layer, you can use “dilated” convolutions:



After 4 layers each neuron has a “context” of 15 time-steps back in the past.

After 5 layers $2^5 - 1 = 31$ time steps.

Question: why not 1 filter of length 31?

In a (1,2,4,8,16) dilation, how many parameters do we save?

Answer: 5 layers x (2x1) filters = 10 instead of 31 parameters.

Dilated 1D causal convolution in Keras

To use time-dilated convolutions, simply use the argument rate `dilation_rate=...` in the `Convolution1D` layer.

```
X,Y = gen_data(noise=0)

modeldil = Sequential()
#<----- Just replaced this block
modeldil.add(Convolution1D(filters=32, kernel_size=ks, padding='causal', dilation_rate=1,
                           batch_input_shape=(None, None, 1)))
modeldil.add(Convolution1D(filters=32, kernel_size=ks, padding='causal', dilation_rate=2))
modeldil.add(Convolution1D(filters=32, kernel_size=ks, padding='causal', dilation_rate=4))
modeldil.add(Convolution1D(filters=32, kernel_size=ks, padding='causal', dilation_rate=8))
#<----- Just replaced this block

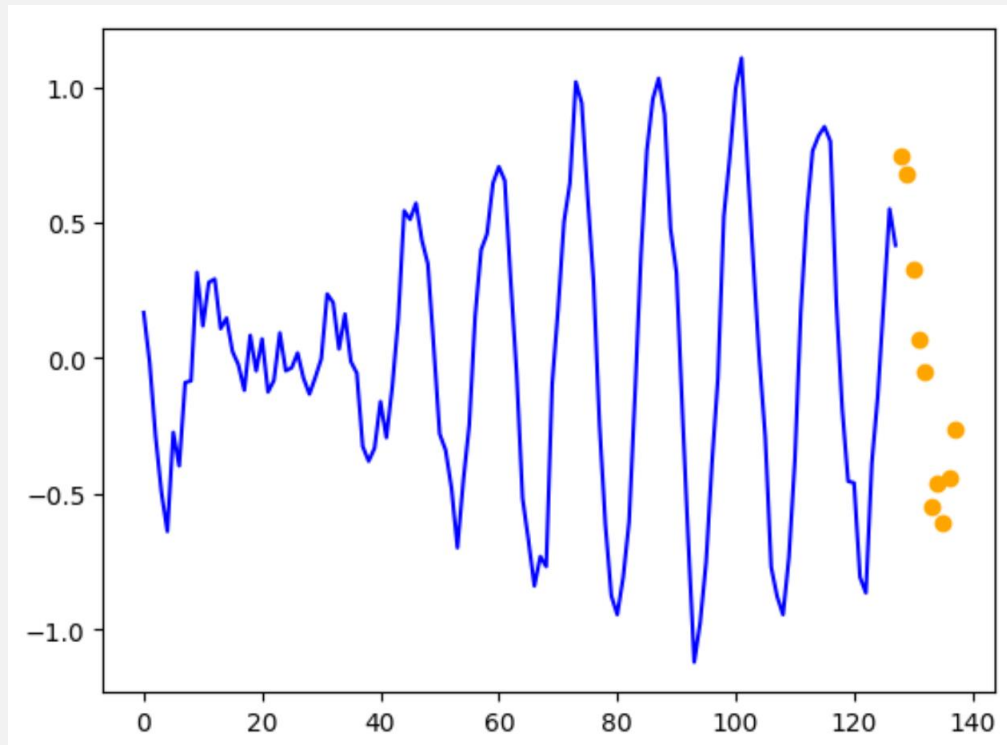
modeldil.add(Dense(1))
modeldil.add(Lambda(slice, arguments={'slice_length':look_ahead}))

modeldil.summary()

modeldil.compile(optimizer='adam',loss='mean_squared_error')

histdil = modeldil.fit(X[0:800], Y[0:800],
                       epochs=200,
                       batch_size=128,
                       validation_data=(X[800:1000],Y[800:1000]), verbose=0)
```

TCN example: time series forecasting



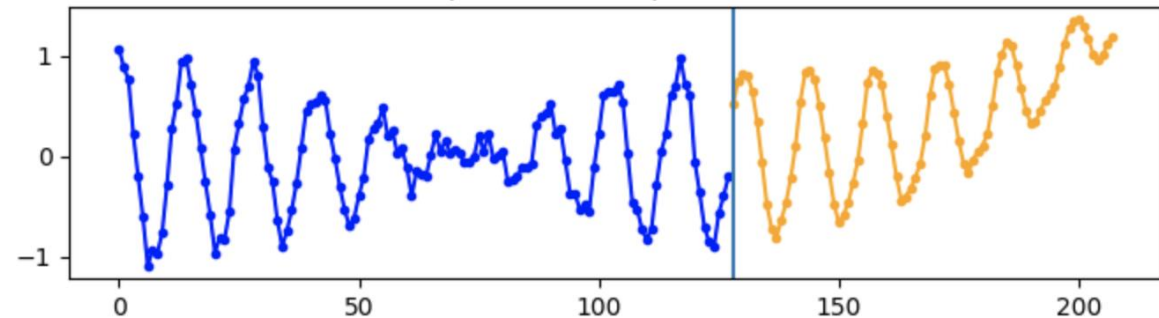
TCNs help if long memory is needed

Dilated 1D CNNs can pick up the long-range time dependencies.

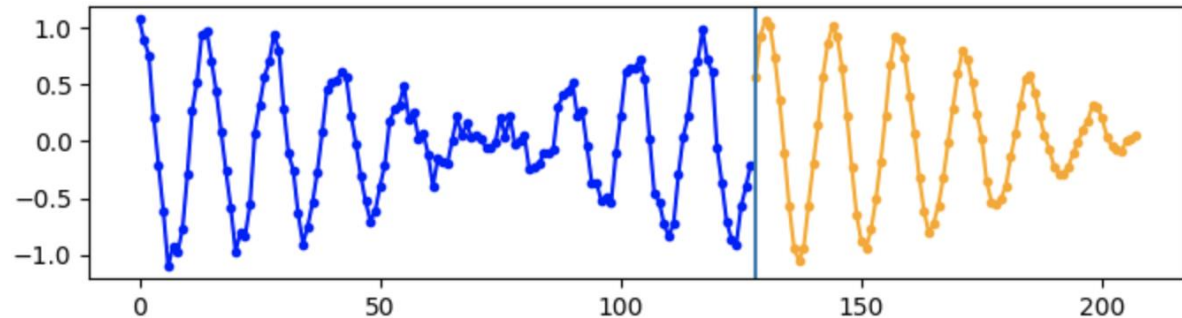
Train data

prediction

1D-CNN



Dilated 1D-CNN



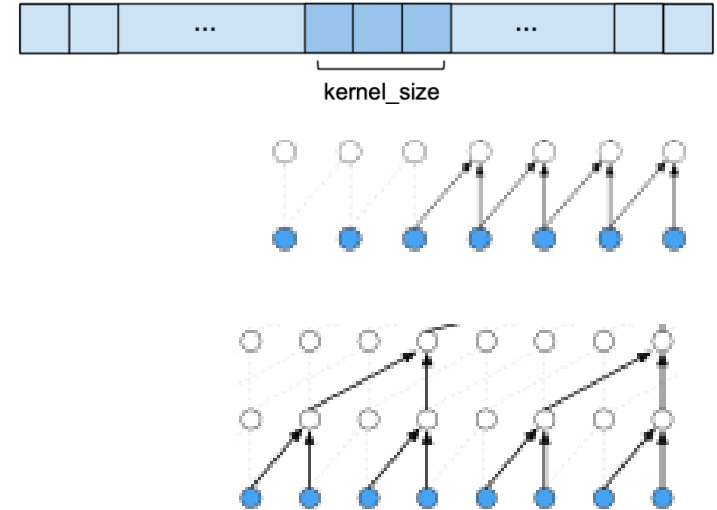
If you want to get a better understanding how 1D convolution work, you can go through the notebook at

https://github.com/tensorchiefs/dl_course_2025/blob/main/notebooks/09_1DConv_sol.ipynb

Summary TCN

- Temporal Convolution Networks:

- 1D-conv layers
- "causal" convolutions
- Dilation
- Residual connections



- TCN can be used in case of sequence data
 - Simpler architecture than LSTM and easier to train.
 - For forecasting application use "causal" variants which only look back
 - Dilation allows to increase the receptive field without needing more weights

From convolutions to attention

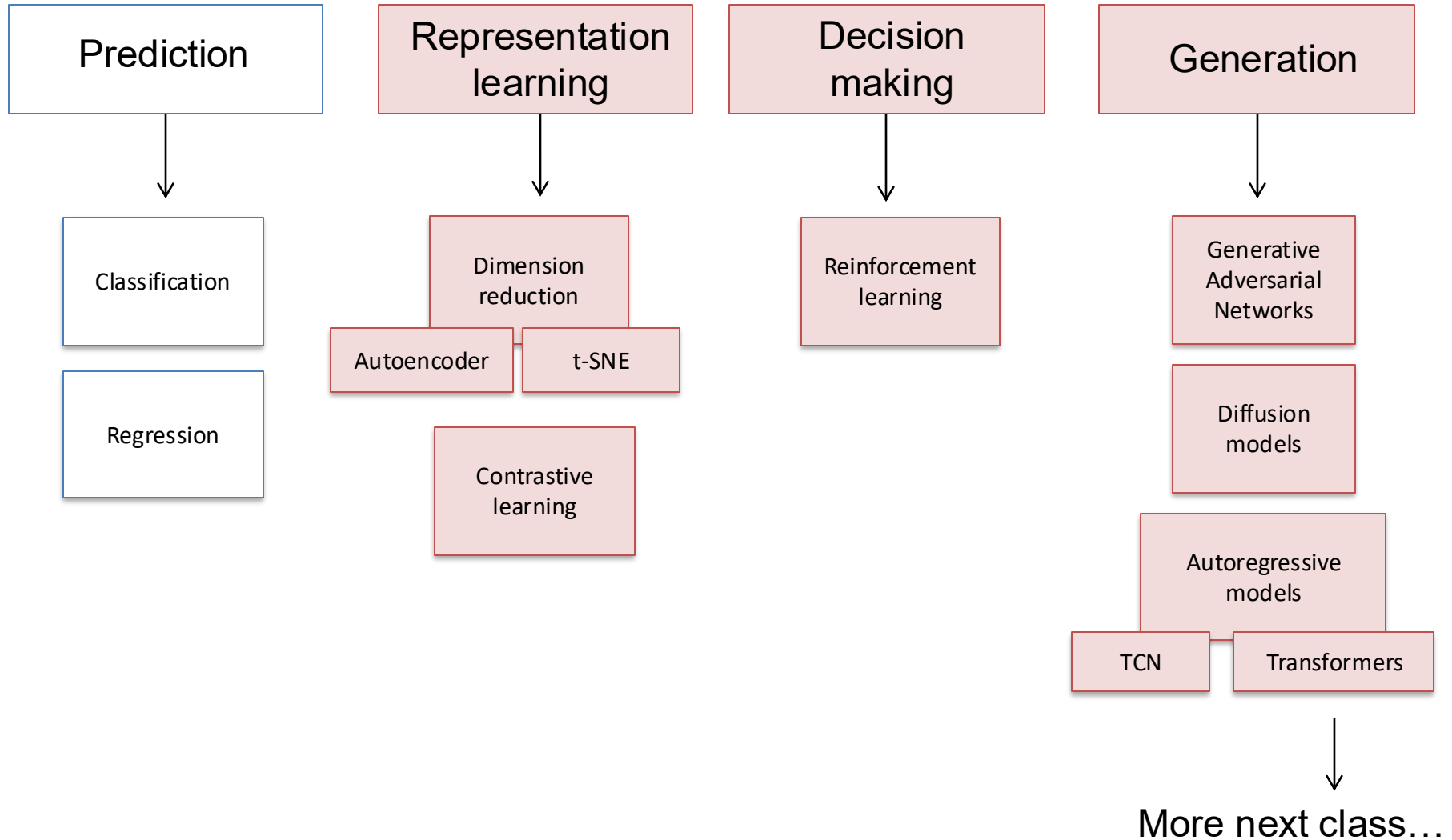
- Task: autoregressive sequence generation

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

- Strengths of TCNs
 - Parallel computation over time.
 - Long-range dependencies via dilations.
 - Stable training (no recurrence).
 - Natural for raw signals (audio, sensor data).
- Limitations of TCNs
 - Fixed receptive field size.
 - Inefficient memory for very long sequences.
 - Architecture must be carefully tuned for sequence length.

➔ Transformers (Attention)

Summary



Hybridization in the zoo

Google's Genie-2

Capabilities

Rapid prototyping

Deploying agents in world models

Model architecture

Responsible development



A cute humanoid robot in the woods.



A humanoid robot in Ancient Egypt.



A first person view of a robot on a purple planet.

Diffusion world model

Genie 2 is an autoregressive [latent diffusion model](#), trained on a large video dataset. After passing through an [autoencoder](#), latent frames from the video are passed to a large [transformer](#) dynamics model, trained with a causal mask similar to that used by large language models.